Sainath Education Trust's
Rajiv Gandhi College of Arts, Commerce & Science
{Permanently Affiliated to University of Mumbai}
Re-Accredited by NAAC, Grade 'B'
Hindi Linguistic Minority Institute

Plot No. 16/17, Sector-10A
Vashi, Navi Mumbai-400703,
Tel.: 2788 0817 / 2766 7377
info@setrgc.edu.in

E-mail: admin@setrgc.edu.in Website: www.setrgc.edu.in

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that **Mr. Rishabdev Muktinath Tripathi** from **T.Y.B. Sc CS** class with seat no.: **319**, has successfully completed the required number of Practical's in the subject **Data Science** as per syllabus laid down by the

University of Mumbai

Academic year 2025-2026

Subject in-charge **Co-ordinator**

_____ _____

External Examiner

_____

Index

Sr. No.	Name of the Practicals	Date	Sign.
1	**Introduction to Excel** • Perform conditional formatting on a dataset using various criteria. • Create a pivot table to analyze and summarize data. • Use VLOOKUP function to retrieve information from a different worksheet or table. • Perform what-if analysis using Goal Seek to determine input values for desired output.	**15/11/2025**	
2	**Data Frames and Basic Data Pre-processing** • Read data from CSV and JSON files into a data frame. • Perform basic data pre-processing tasks such as handling missing values and outliers. • Manipulate and transform data using functions like filtering, sorting, and grouping.	**22/11/2025**	
3	**Feature Scaling and Dummification** • Apply feature-scaling techniques like standardization and normalization to numerical features. • Perform feature dummification to convert categorical variables into numerical representations.	**29/11/2025**	
4	**Hypothesis Testing** • Formulate null and alternative hypotheses for a given problem. • Conduct a hypothesis test using appropriate statistical tests (e.g., t-test, chi- square test). • Interpret the results and draw conclusions based on the test outcomes.	**06/12/2025**	
5	**ANOVA (Analysis of Variance)** • Perform one-way ANOVA to compare means across multiple groups. • Conduct post-hoc tests to identify significant differences between group means.	**20/12/2025**	
6	**Regression and Its Types** • Implement simple linear regression using a dataset. • Explore and interpret the regression model coefficients and goodness-of-fit measures. • Extend the analysis to multiple linear regression and assess the impact of additional predictors.	**03/01/2026**	

Sr. No.	Name of the Practicals	Date	Sign.
7	**Logistic Regression and Decision Tree** • Build a logistic regression model to predict a binary outcome. • Evaluate the model's performance using classification metrics (e.g., accuracy, precision, recall). • Construct a decision tree model and interpret the decision rules for classification.	**10/01/2026**	
8	**K-Means Clustering** • Apply the K-Means algorithm to group similar data points into clusters. • Determine the optimal number of clusters using elbow method or silhouette analysis. • Visualize the clustering results and analyze the cluster characteristics.	**24/01/2026**	
9	**Principal Component Analysis (PCA)** • Perform PCA on a dataset to reduce dimensionality. • Evaluate the explained variance and select the appropriate number of principal components. • Visualize the data in the reduced-dimensional space.	**07/02/2026**	
10	**Data Visualization and Storytelling** • Create meaningful visualizations using data visualization tools. • Combine multiple visualizations to tell a compelling data story. • Present the findings and insights in a clear and concise manner.	**14/02/2026**	

Practical 1

Aim: Introduction to Excel

- Perform conditional formatting on a dataset using various criteria.
- Create a pivot table to analyze and summarize data.
- Use VLOOKUP function to retrieve information from a different worksheet or table.
- Perform what-if analysis using Goal Seek to determine input values for desired output.

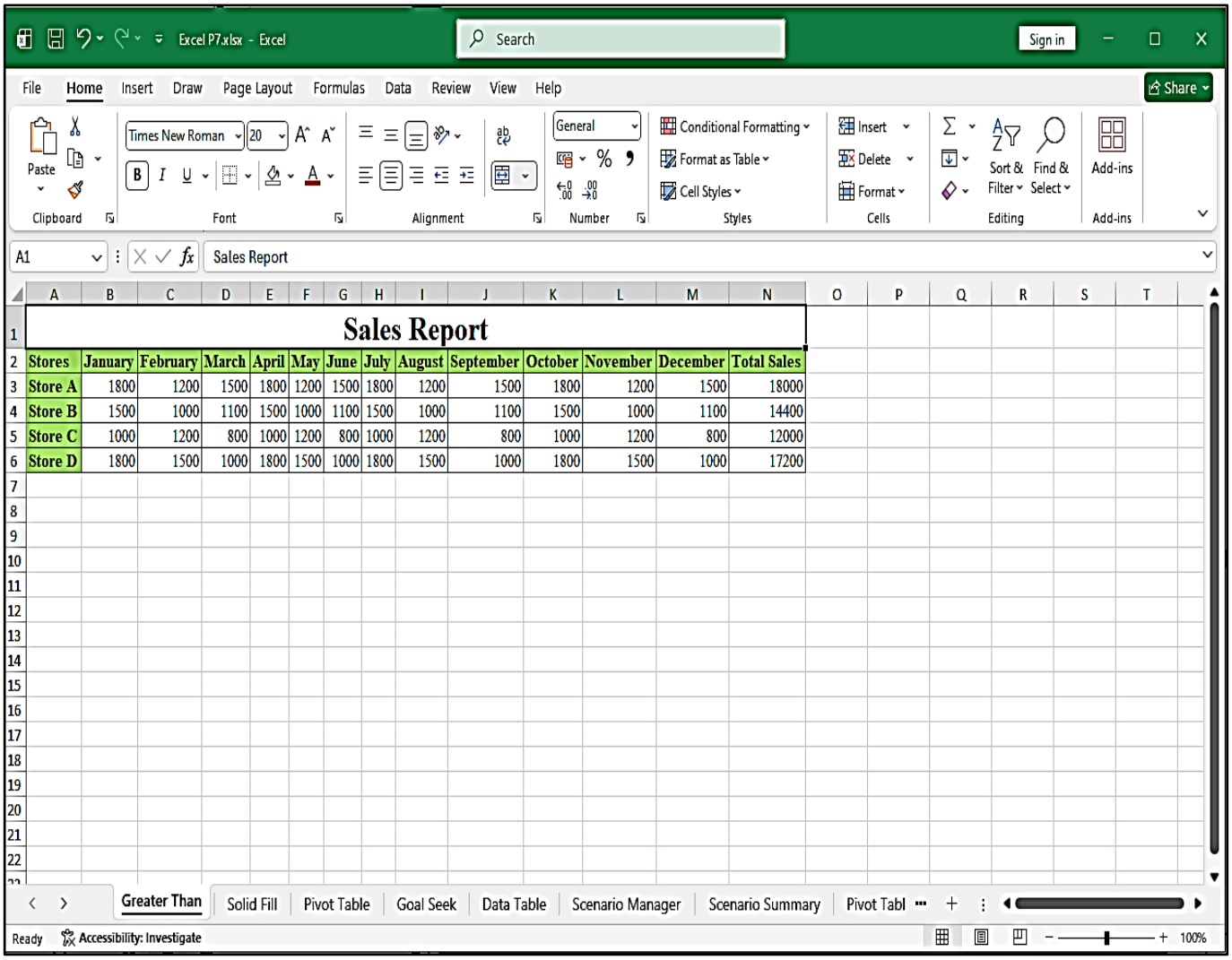
Conditional Formatting

Conditional Formatting is an Excel feature that automatically changes the appearance of cells (color, font, icons, data bars) based on specific conditions or rules. It helps users quickly identify patterns, trends, or outliers in data without manually scanning values. It includes different appearance or highlights with rules like Highlight Cells Rules, Top/Bottom Rules, Data Bars, Color Scales, Icon Sets.

Home Tab → Styles Group → Conditional Formatting

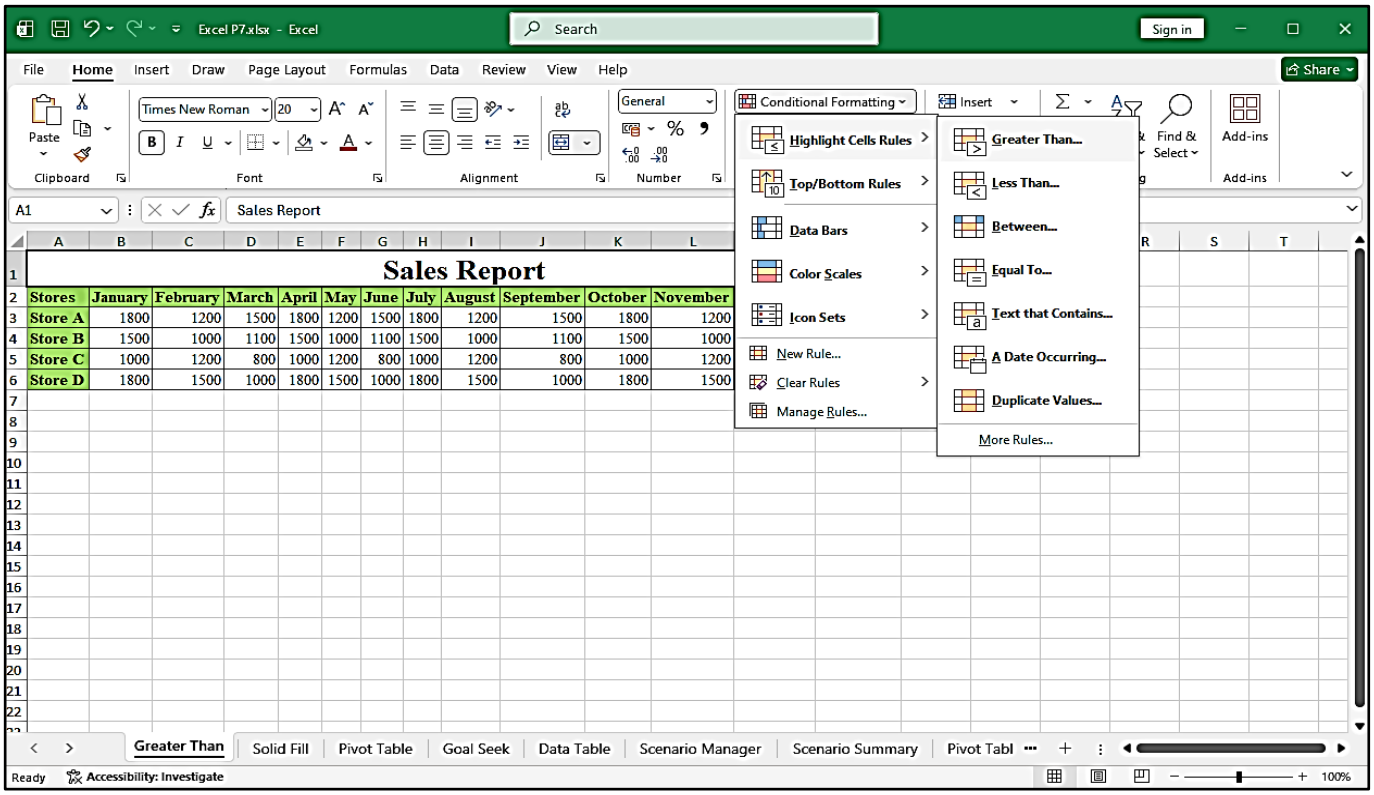
Steps

Step 1 : Go to conditional formatting > Greater Than

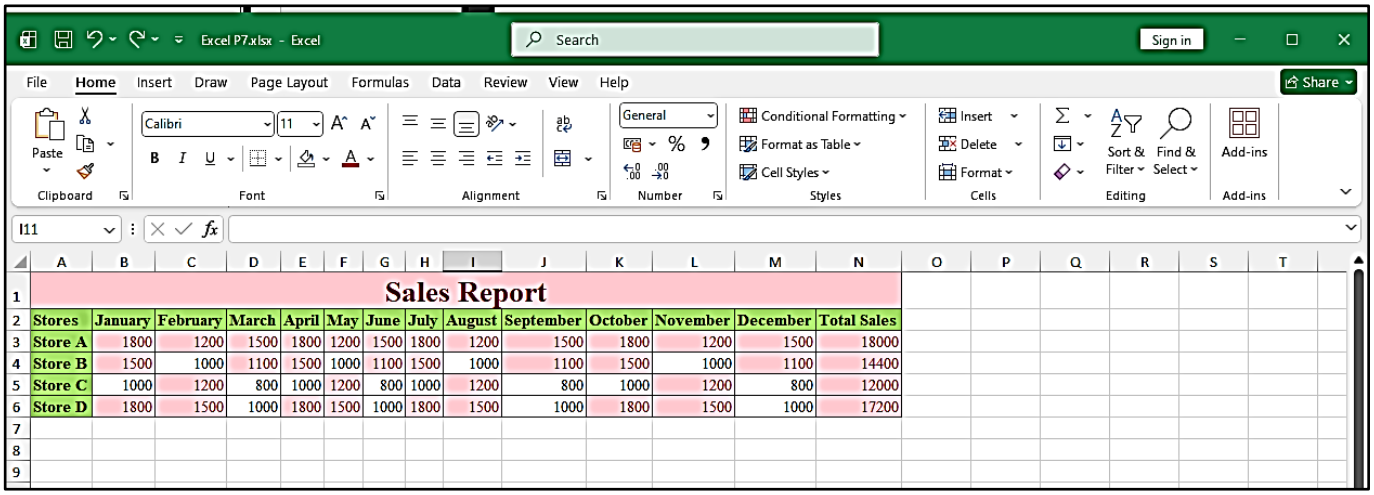


Sales Report												
Stores	January	February	March	April	May	June	July	August	September	October	November	December	Total Sales
Store A	1800	1200	1500	1800	1200	1500	1800	1200	1500	1800	1200	1500	18000
Store B	1500	1000	1100	1500	1000	1100	1500	1000	1100	1500	1000	1100	14400
Store C	1000	1200	800	1000	1200	800	1000	1200	800	1000	1200	800	12000
Store D	1800	1500	1000	1800	1500	1000	1800	1500	1000	1800	1500	1000	17200

Step 2: Enter the greater than filter value for example 1000.

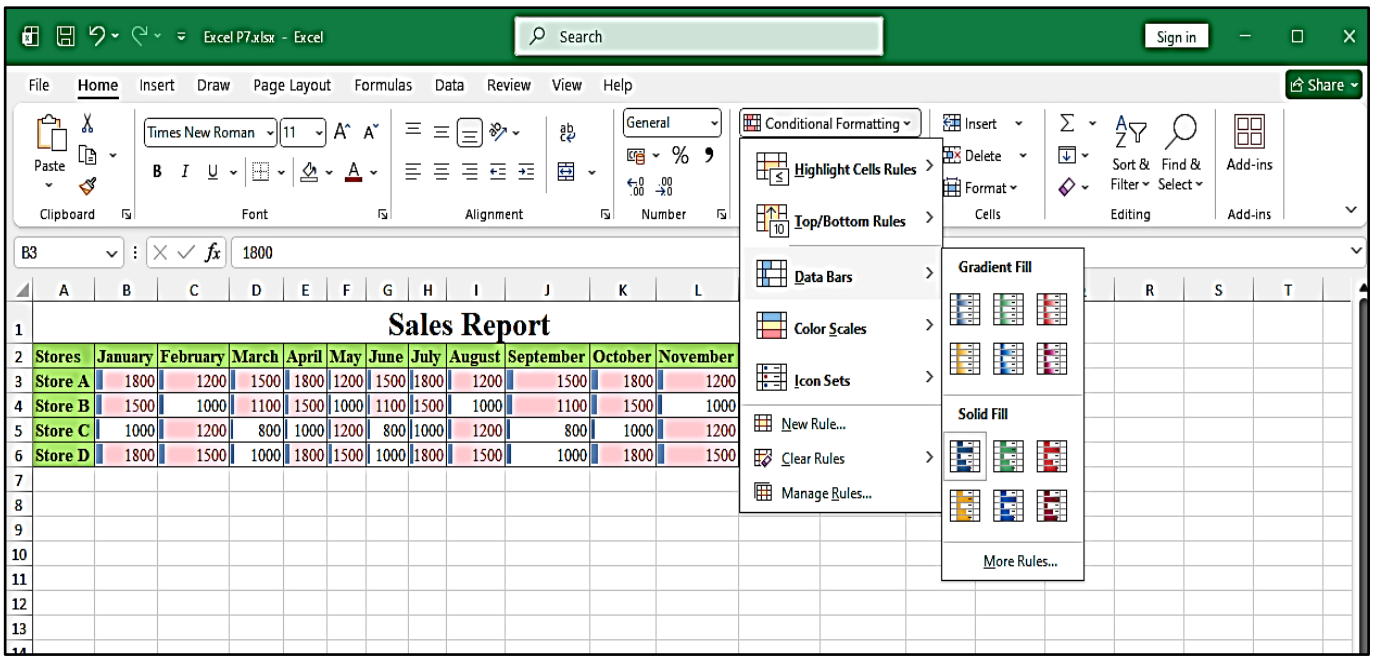


Sales Report											
Stores	January	February	March	April	May	June	July	August	September	October	November
Store A	1800	1200	1500	1800	1200	1500	1800	1200	1500	1800	1200
Store B	1500	1000	1100	1500	1000	1100	1500	1000	1100	1500	1000
Store C	1000	1200	800	1000	1200	800	1000	1200	800	1000	1200
Store D	1800	1500	1000	1800	1500	1000	1800	1500	1000	1800	1500



Sales Report													
Stores	January	February	March	April	May	June	July	August	September	October	November	December	Total Sales
Store A	1800	1200	1500	1800	1200	1500	1800	1200	1500	1800	1200	1500	18000
Store B	1500	1000	1100	1500	1000	1100	1500	1000	1100	1500	1000	1100	14400
Store C	1000	1200	800	1000	1200	800	1000	1200	800	1000	1200	800	12000
Store D	1800	1500	1000	1800	1500	1000	1800	1500	1000	1800	1500	1000	17200

Step 3: Go to Data Bars > Solid Fill in conditional formatting.



Sales Report											
Stores	January	February	March	April	May	June	July	August	September	October	November
Store A	1800	1200	1500	1800	1200	1500	1800	1200	1500	1800	1200
Store B	1500	1000	1100	1500	1000	1100	1500	1000	1100	1500	1000
Store C	1000	1200	800	1000	1200	800	1000	1200	800	1000	1200
Store D	1800	1500	1000	1800	1500	1000	1800	1500	1000	1800	1500

Sales Report

Stores	January	February	March	April	May	June	July	August	September	October	November	December	Total Sales
Store A	1800	1200	1500	1800	1200	1500	1800	1200	1500	1800	1200	1500	18000
Store B	1500	1000	1100	1500	1000	1100	1500	1000	1100	1500	1000	1100	14400
Store C	1000	1200	800	1000	1200	800	1000	1200	800	1000	1200	800	12000
Store D	1800	1500	1000	1800	1500	1000	1800	1500	1000	1800	1500	1000	17200

Pivot Table

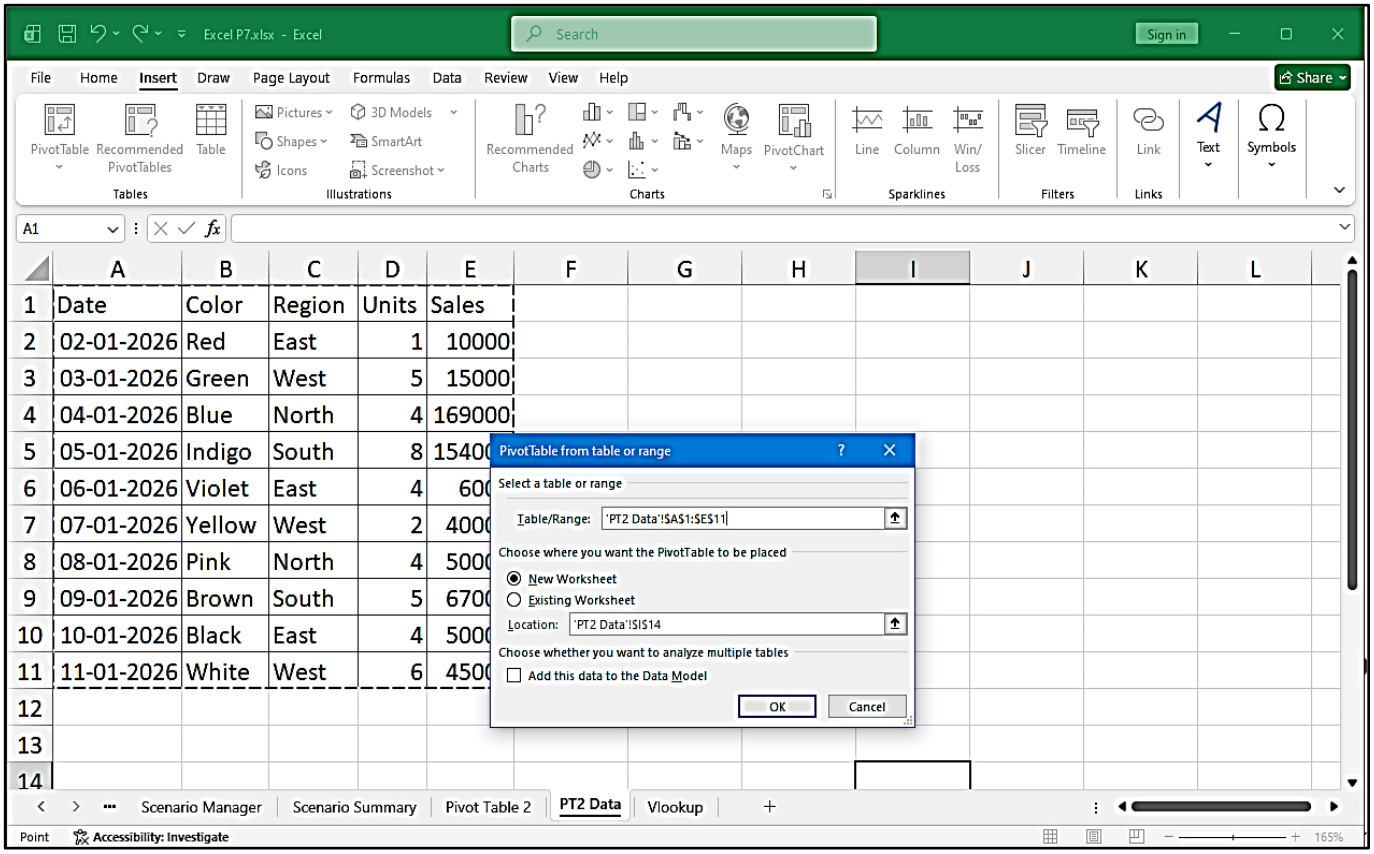
A Pivot Table is a powerful Excel tool used to summarize, analyze, and reorganize large datasets by grouping data and performing calculations like sum, count, average, max, or min with just drag-and-drop actions.
Insert Tab → Tables Group → PivotTable

Steps

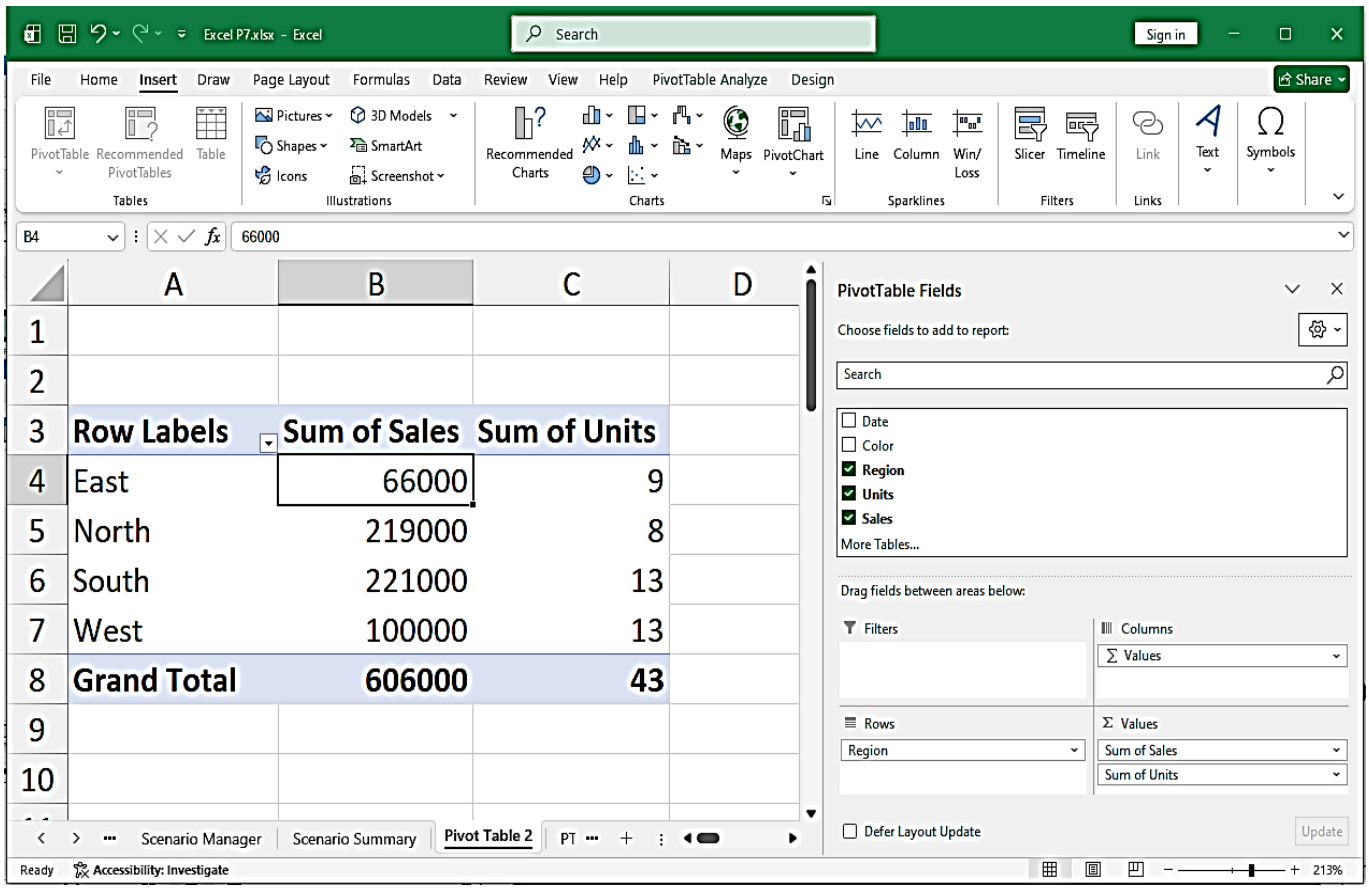
Step 1: First Create the Excel Data and select the entire table and go to Insert tab PivotChart > Pivotchart

	A	B	C	D	E
1	Date	Color	Region	Units	Sales
2	03-Jan-20	Red	West	1	110000
3	14-Jan-20	blue	South	8	96000
4	21-Jan-20	green	west	2	26000
5	30-Jan-20	blue	north	7	84000
6	07-Feb-20	green	north	8	25000
7	13-Feb-20	red	south	2	60000
8	22-Feb-20	blue	east	5	35000
9	01-Mar-20	green	west	2	87000
10	13-Mar-20	blue	east	8	69000
11	23-Mar-20	blue	north	7	54000
12					

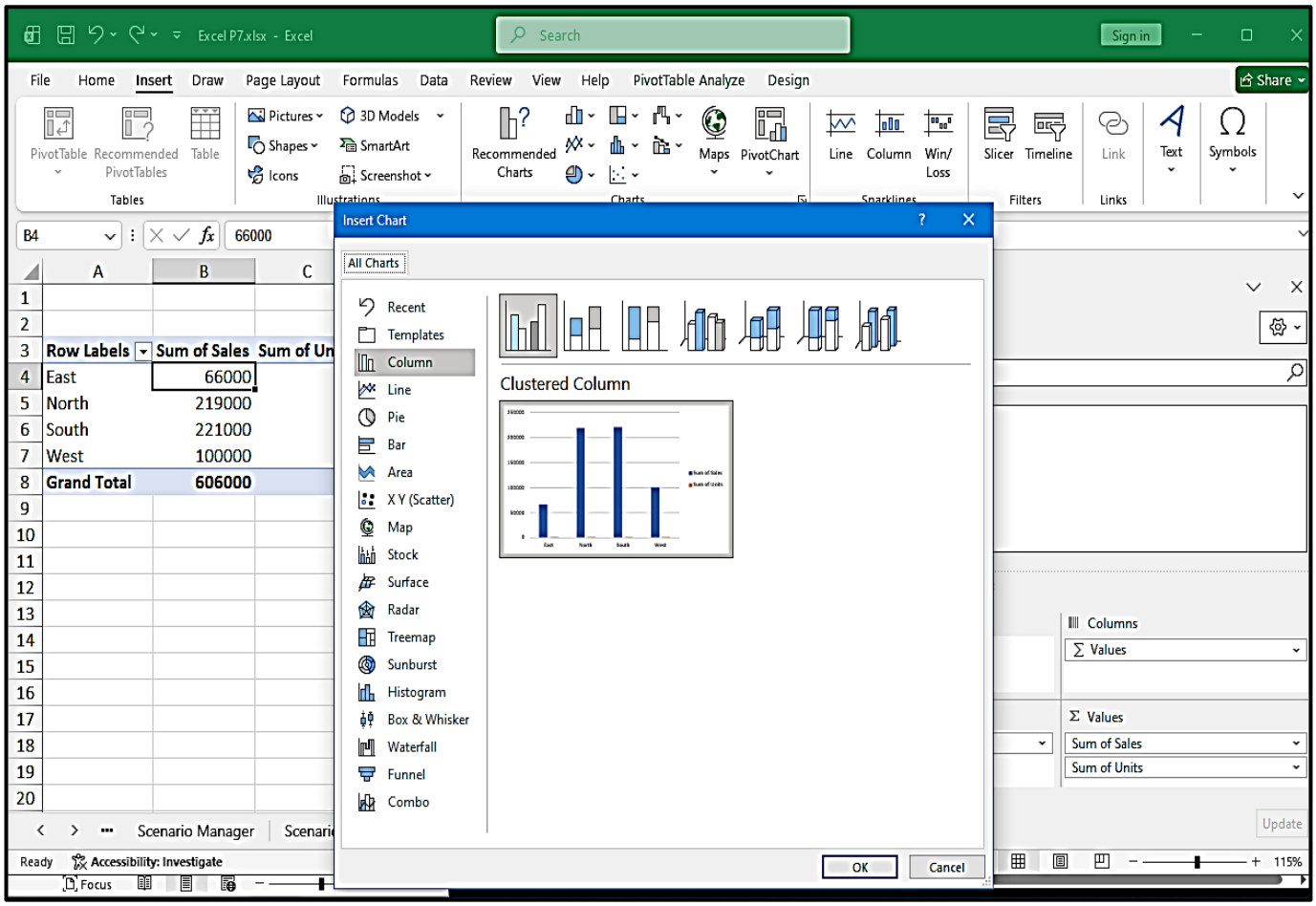
Step 2: Select the pivot chart and pivot table option from insert tab. Select "New worksheet" in the create pivot chart window. Select the table range and cell where you want the output.

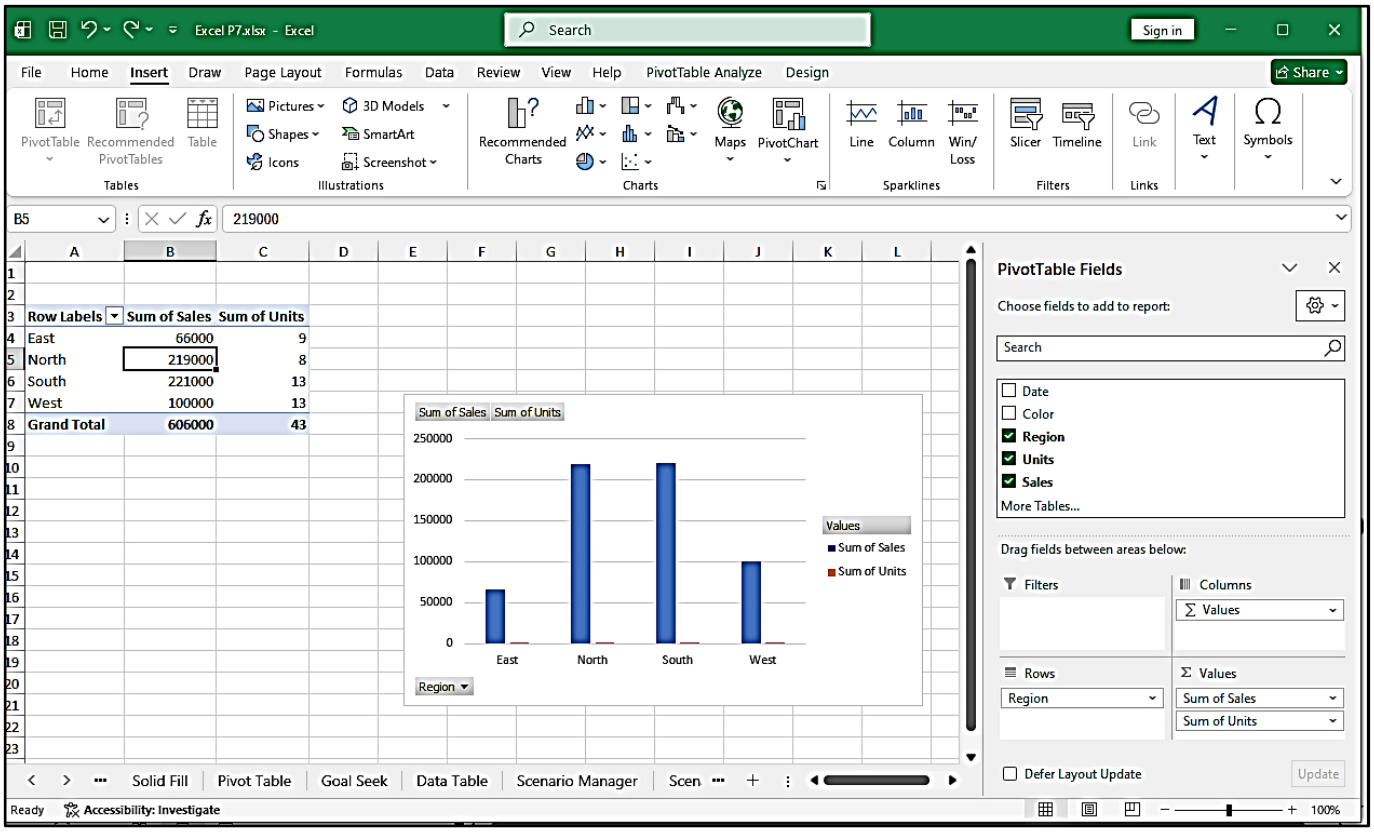


Step 3: Select and drag attributes in the below boxes.



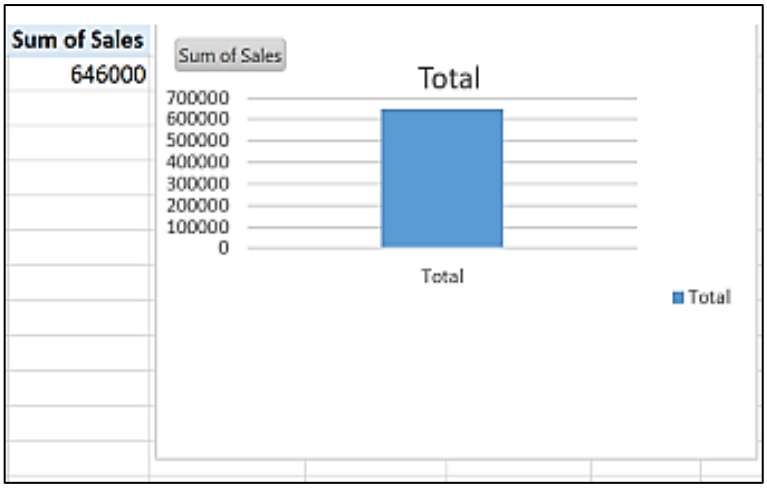
Step 4 : Select PivotChart from PivotTable Analyze Tab. Select the Chart Type and click ok.



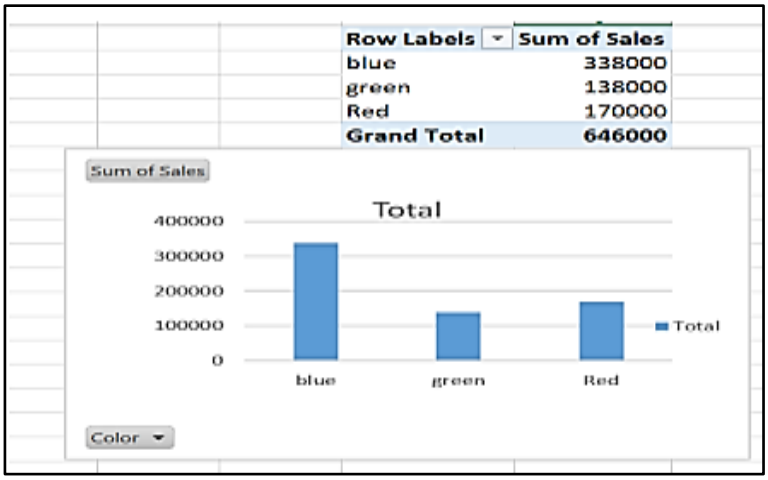


Step 5 : Perform the following Questions.

i) Find out the total sales.

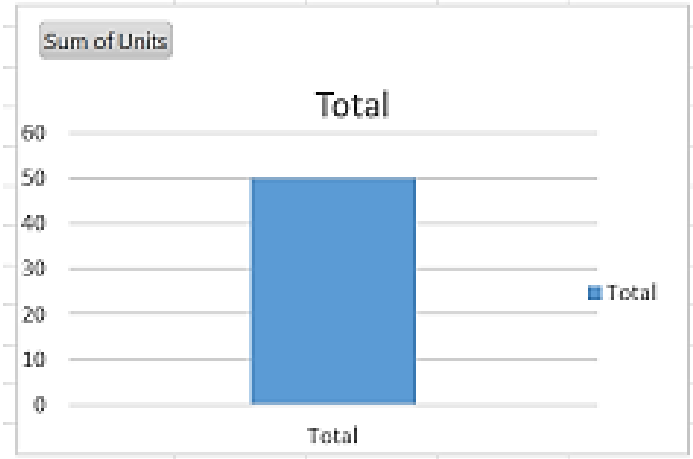


ii) Find out the sum of sales Color wise



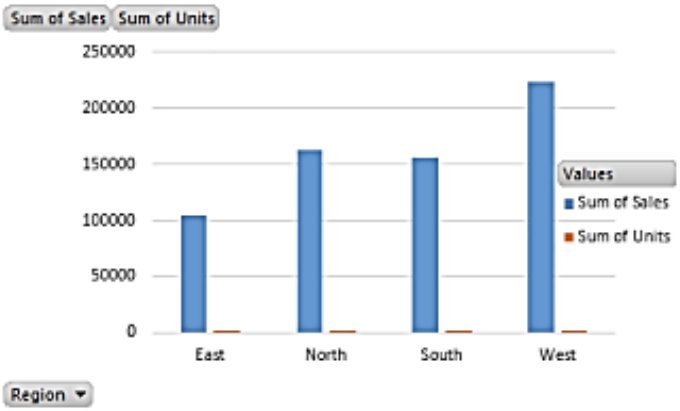
iii) **Find out the sum of units**

Sum of Units
50



iv) **Find out Region wise total sales and total units**

Row Labels	Sum of Sales	Sum of Units
east	104000	13
north	163000	22
South	156000	10
West	223000	5
Grand Total	**646000**	**50**



VLOOKUP

VLOOKUP (Vertical Lookup) is a function used to search for a value in the first column of a table and return a corresponding value from another column in the same row.

Formulas Tab → Function Library → Lookup & Reference → VLOOKUP

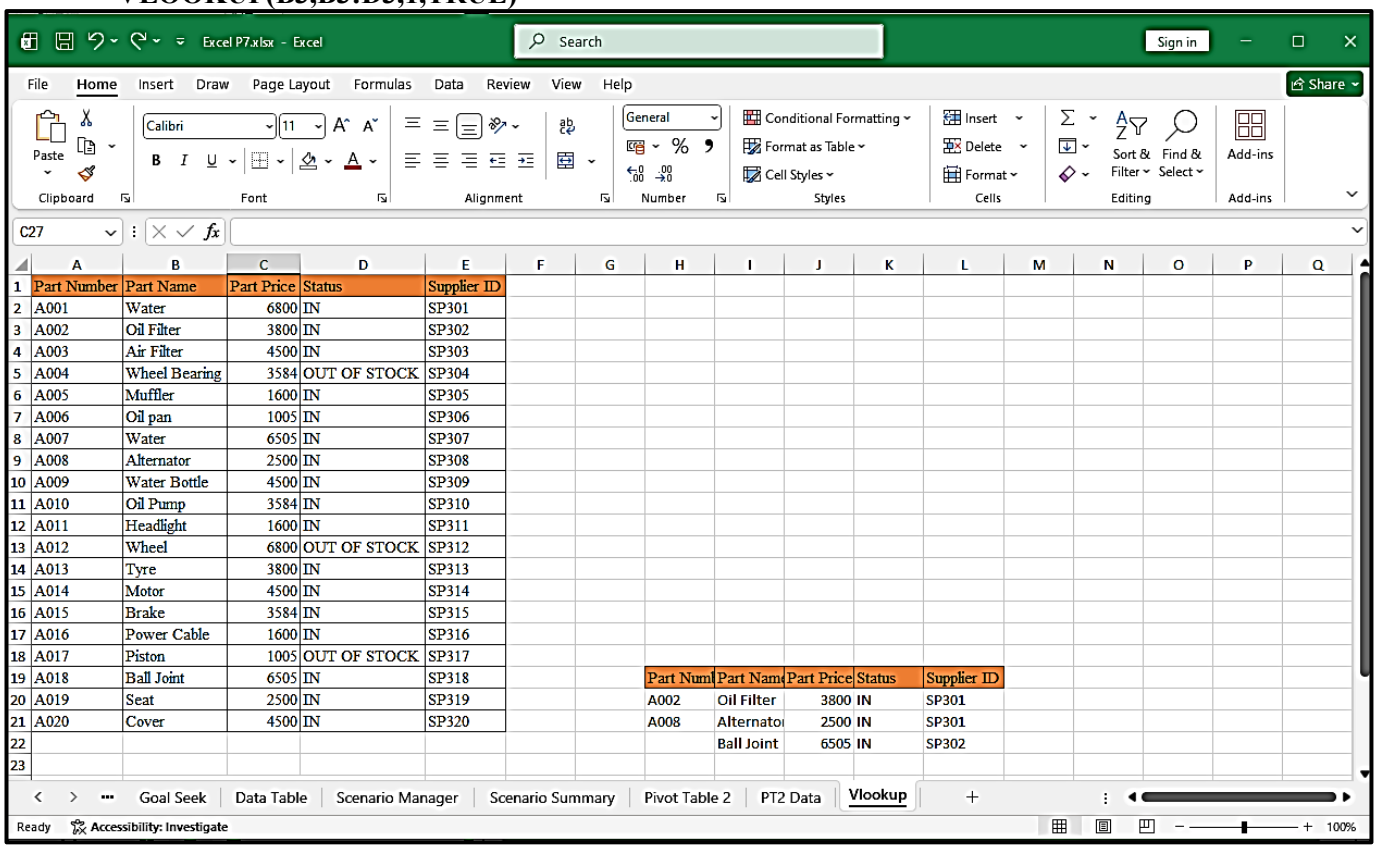
Formula :

=VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup])

Steps

Step l : click on an empty cell and type the following command:

=VLOOKUP(B3,B3:D3,1,TRUE)



Part Number	Part Name	Part Price	Status	Supplier ID
A001	Water	6800	IN	SP301
A002	Oil Filter	3800	IN	SP302
A003	Air Filter	4500	IN	SP303
A004	Wheel Bearing	3584	OUT OF STOCK	SP304
A005	Muffler	1600	IN	SP305
A006	Oil pan	1005	IN	SP306
A007	Water	6505	IN	SP307
A008	Alternator	2500	IN	SP308
A009	Water Bottle	4500	IN	SP309
A010	Oil Pump	3584	IN	SP310
A011	Headlight	1600	IN	SP311
A012	Wheel	6800	OUT OF STOCK	SP312
A013	Tyre	3800	IN	SP313
A014	Motor	4500	IN	SP314
A015	Brake	3584	IN	SP315
A016	Power Cable	1600	IN	SP316
A017	Piston	1005	OUT OF STOCK	SP317
A018	Ball Joint	6505	IN	SP318
A019	Seat	2500	IN	SP319
A020	Cover	4500	IN	SP320

Part Numl	Part Nam	Part Price	Status	Supplier ID
A002	Oil Filter	3800	IN	SP301
A008	Alternato	2500	IN	SP301
	Ball Joint	6505	IN	SP302

i) Find the part name of part number “A002”

=VLOOKUP(B3,B2:E21,2,FALSE)

ii) Find the Supplier ID for part name “Ball Joint”

=VLOOKUP(“Ball joint”,C2:F21,4,FALSE)

iii) Find the Part price for part name “Muffler”

=VLOOKUP(“Muffler”,C2:E21,2,FALSE)

iv) Find the Status of Part number “A008”

=VLOOKUP(B9,B2:E21,4,FALSE)

1	Find the Part Name of A002	Alternator
2	Find the supplier id for Ball Joint	SP318
3	Find out the part price for part name muffler	1600
4	Find the status of part number A008	OUT OF STOCK

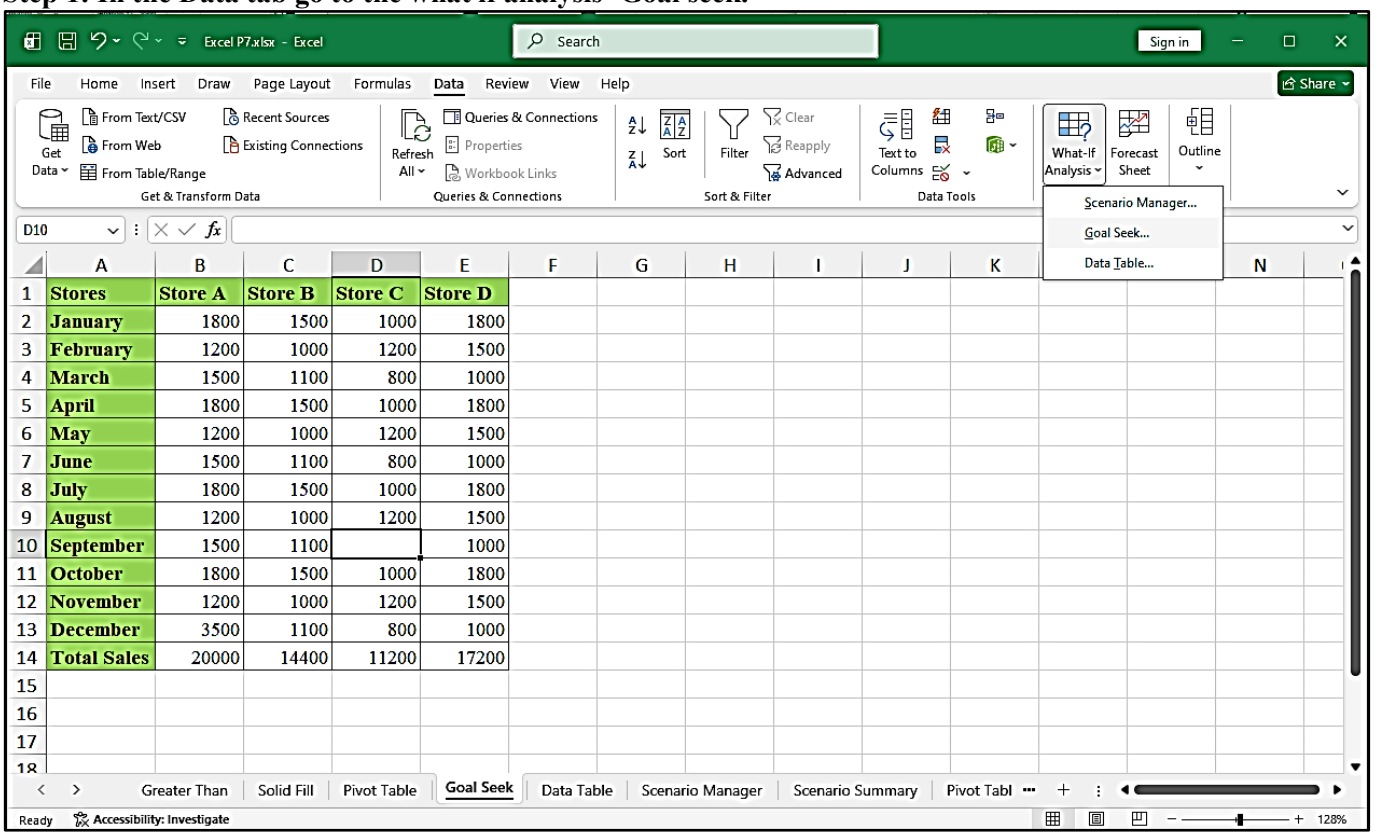
What-if-Analysis using Goal Seek

Goal Seek is a What-If Analysis tool that finds the required input value to achieve a desired result in a formula. It works by changing one variable until the target output is reached.

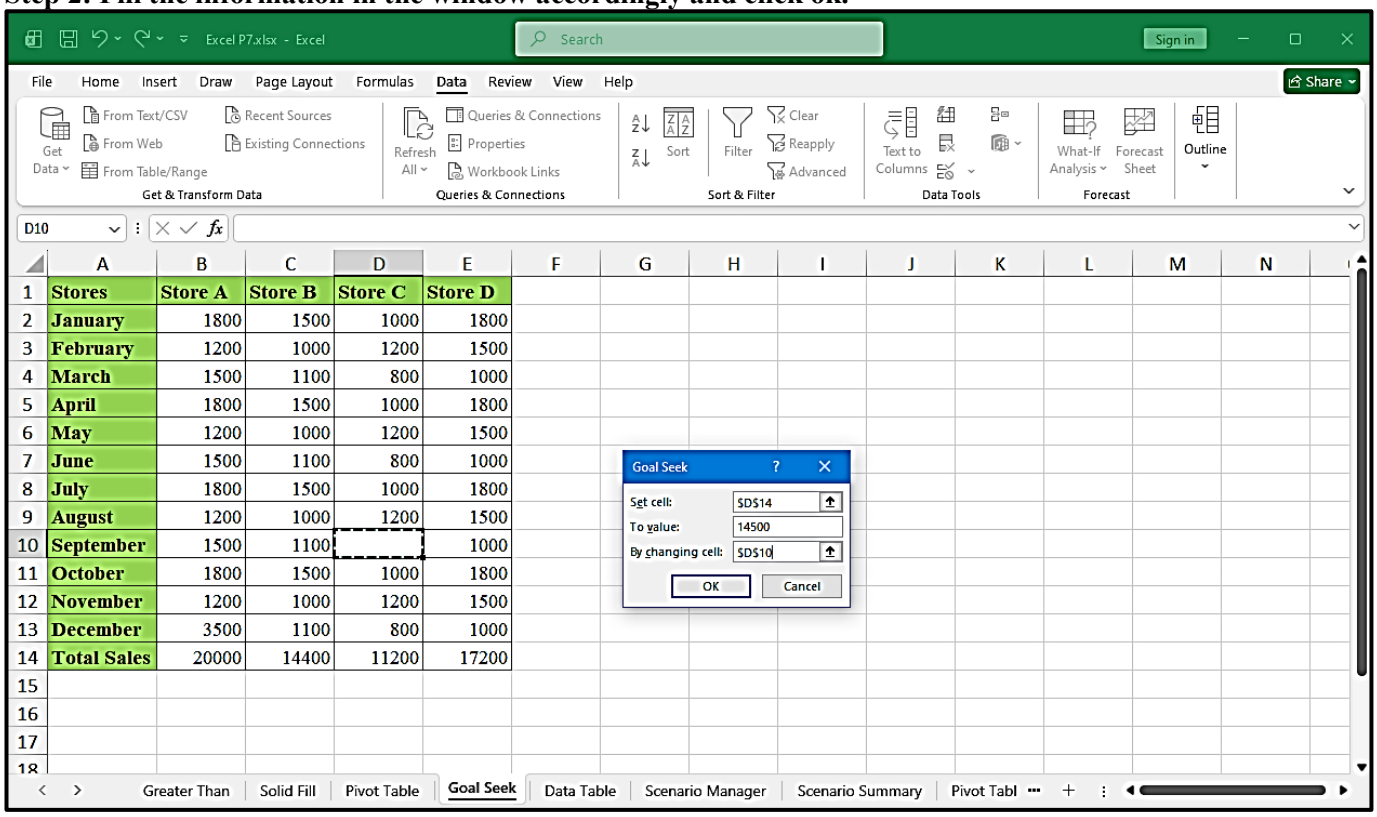
Data Tab → Forecast Group → What-If Analysis → Goal Seek

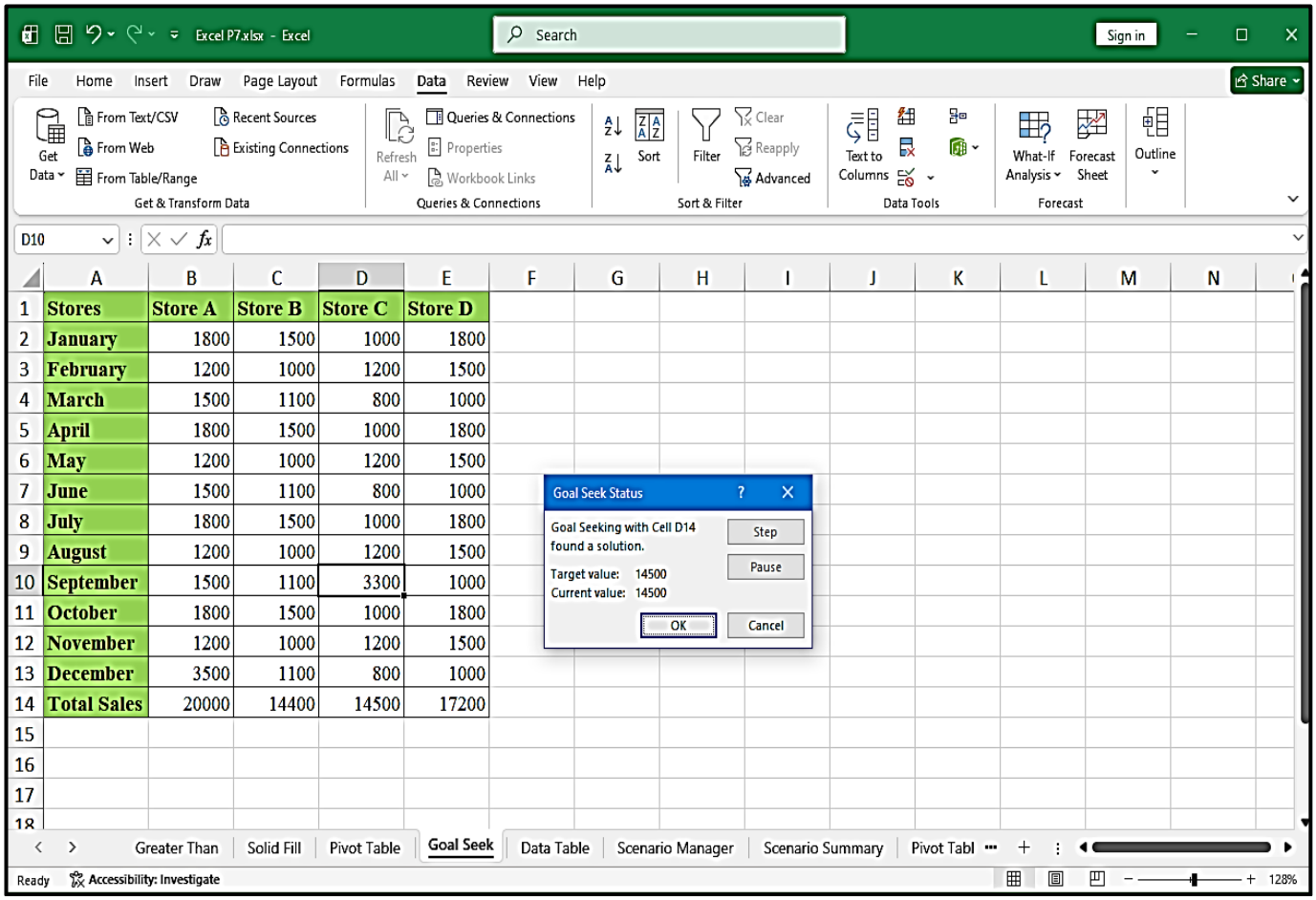
Steps

Step 1: In the Data tab go to the what if analysis>Goal seek.



Step 2: Fill the information in the window accordingly and click ok.





	A	B	C	D	E
1	Stores	Store A	Store B	Store C	Store D
2	January	1800	1500	1000	1800
3	February	1200	1000	1200	1500
4	March	1500	1100	800	1000
5	April	1800	1500	1000	1800
6	May	1200	1000	1200	1500
7	June	1500	1100	800	1000
8	July	1800	1500	1000	1800
9	August	1200	1000	1200	1500
10	September	1500	1100	3300	1000
11	October	1800	1500	1000	1800
12	November	1200	1000	1200	1500
13	December	3500	1100	800	1000
14	Total Sales	20000	14400	14500	17200

Practical 2

Aim: Data Frames and Basic Data Pre-processing

- Read data from CSV and JSON files into a data frame.
- Perform basic data pre-processing tasks such as handling missing values and outliers.
- Manipulate and transform data using functions like filtering, sorting, and grouping.

A **DataFrame** is a two-dimensional, labeled data structure used in data analysis, mainly in Pandas (Python). It looks like an Excel sheet or SQL table, with rows and columns, where each column can store different data types (int, float, string, etc.).

Data Pre-processing is the process of cleaning, transforming, and organizing raw data to make it suitable for analysis or machine learning models. Raw data is messy → preprocessing makes it usable.

Functions:

Filtering Data → Selecting specific rows based on conditions.

Sorting Data → Arranging data in ascending or descending order.

Grouping Data → Grouping data based on a column and applying aggregation functions like sum, mean, count.

Input:

```
import pandas as pd
import numpy as np
# Reading a CSV file into a DataFrame
df=pd.read_csv(r"Iris.csv")
print(df.head()) # Display the first 5 rows of the DataFrame
```

Output:

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

Input:

```
# Step 2: Basic Data Exploration
df.head()
```

Output:

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

Input:

```
df.tail()
```

Output:

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            Species
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica
```

Input:

```
df.describe()
```

Output:

```
                Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  150.000000     150.000000    150.000000     150.000000    150.000000
mean    75.500000       5.843333      3.054000       3.758667      1.198667
std     43.445368       0.828066      0.433594       1.764420      0.763161
min      1.000000       4.300000      2.000000       1.000000      0.100000
25%     38.250000       5.100000      2.800000       1.600000      0.300000
50%     75.500000       5.800000      3.000000       4.350000      1.300000
75%    112.750000       6.400000      3.300000       5.100000      1.800000
max    150.000000       7.900000      4.400000       6.900000      2.500000
```

Input:

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

Input:

```
df.shape
```

Output:

```
(150, 6)
```

Input:

```
df.dtypes
```

Output:

```
Id                 int64
SepalLengthCm    float64
SepalWidthCm     float64
PetalLengthCm    float64
PetalWidthCm     float64
Species           object
dtype: object
```

Input:

```
df.columns
```

Output:

```
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')
```

Input:

```
# Step 3: Checking for Missing Values
# Checking for missing values in each column of the CSV DataFrame
missing_values = df.isnull().sum()
print("\nMissing Values in CSV Data:")
print(missing_values)
```

Output:

```
Missing Values in CSV Data:
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
```

Input:

```
df = df.drop(columns=['Species'])
# Step 4: Handling Missing Values
# We will fill missing values in columns with the mean of the column
# (You could also drop missing rows or use other strategies depending on your need
fill= df.fillna(df.mean())
print("\nFilled Missing Values with Mean (CSV Data):")
print(df.head())
```

Output:

```
Filled Missing Values with Mean (CSV Data):
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0   1            5.1           3.5            1.4           0.2
1   2            4.9           3.0            1.4           0.2
2   3            4.7           3.2            1.3           0.2
3   4            4.6           3.1            1.5           0.2
4   5            5.0           3.6            1.4           0.2
```

Input:

```
# Step 5: Handling Outliers
# Here we will calculate Z-scores and remove rows where Z-score is greater than 3
z_scores = np.abs((fill - fill.mean()) / fill.std())
z_scores
```

Output:

```
           Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0    1.714797       0.897674      1.028611       1.336794      1.308593
1    1.691780       1.139200      0.124540       1.336794      1.308593
2    1.668762       1.380727      0.336720       1.393470      1.308593
3    1.645745       1.501490      0.106090       1.280118      1.308593
4    1.622728       1.018437      1.259242       1.336794      1.308593
..        ...            ...           ...            ...           ...
145  1.622728       1.034539      0.124540       0.816888      1.443121

146  1.645745       0.551486      1.277692       0.703536      0.918985
147  1.668762       0.793012      0.124540       0.816888      1.050019
148  1.691780       0.430722      0.797981       0.930239      1.443121
149  1.714797       0.068433      0.124540       0.760212      0.787951
```

Input:

```
# Remove rows where any Z-score is greater than 3 (outliers)
do = fill[(z_scores < 3).all(axis=1)]
print("\nData After Removing Outliers (CSV Data):")
print(do.head())
```

Output:

```
Data After Removing Outliers (CSV Data):
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0   1            5.1           3.5            1.4           0.2
1   2            4.9           3.0            1.4           0.2
2   3            4.7           3.2            1.3           0.2
3   4            4.6           3.1            1.5           0.2
4   5            5.0           3.6            1.4           0.2
```

Input:

```
# Step 6: Filtering Data (Example: Select rows where a column value is greater tha
threshold_value = 3 # Example threshold value
filter = do[do['SepalLengthCm'] > threshold_value]
print(f"\nFiltered Data (Rows with column_name > {threshold_value}):")
print(filter)
```

Output:

```
Filtered Data (Rows with column_name > 3):
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   ...            ...           ...            ...           ...
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

[149 rows x 5 columns]
```

Input:

```
# Step 7: Sorting Data (Sorting by a column in descending order)
df_sorted = filter.sort_values(by='SepalWidthCm', ascending=False)
print(df_sorted.head())
```

Output:

```
     Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
33   34            5.5           4.2            1.4           0.2
32   33            5.2           4.1            1.5           0.1
14   15            5.8           4.0            1.2           0.2
5     6            5.4           3.9            1.7           0.4
16   17            5.4           3.9            1.3           0.4
```

Input:

```
# Step 8: Grouping Data (Example: Group by a column and calculate the mean of anot
df_grouped = df_sorted.groupby('SepalLengthCm').agg({
 'PetalLengthCm': 'mean', # Calculate the mean of 'another_column' for each gr
 'PetalWidthCm': 'sum' # Calculate the sum of 'yet_another_column' for each gr
}).reset_index() # Reset index to avoid multi-index
print("\nGrouped Data (Mean and Sum for Each Group):")
print(df_grouped.head())
```

Output:

```
Grouped Data (Mean and Sum for Each Group):
   SepalLengthCm  PetalLengthCm  PetalWidthCm
0            4.3       1.100000           0.1
1            4.4       1.333333           0.6
2            4.5       1.300000           0.3
3            4.6       1.325000           0.9
4            4.7       1.450000           0.4
```

Practical 3

Aim: Feature Scaling and Dummification

- Apply feature-scaling techniques like standardization and normalization to numerical features.
- Perform feature dummification to convert categorical variables into numerical representations.

Feature scaling is a data preprocessing technique used to bring all numerical features to a common scale so that no variable dominates the model due to large values. **Standardization** transforms data to have mean 0 and standard deviation 1, while **Normalization** rescales values between 0 and 1. It is essential for distance-based and gradient-based algorithms. Feature scaling improves model accuracy, convergence speed, and performance.

Dummification is the process of converting categorical variables into numerical form so that machine learning algorithms can process them. It is commonly done using one-hot encoding, where each category is represented as a binary column (0 or 1). This avoids giving false ordinal meaning to categories. Dummification is essential before applying most ML models.

Input:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler
df = pd.read_csv(r"wine.csv")
df
```

Output:

```
     Wine  Alcohol  Malic.acid   Ash   Acl   Mg  Phenols  Flavanoids  \
0       1    14.23        1.71  2.43  15.6  127     2.80        3.06
1       1    13.20        1.78  2.14  11.2  100     2.65        2.76
2       1    13.16        2.36  2.67  18.6  101     2.80        3.24
3       1    14.37        1.95  2.50  16.8  113     3.85        3.49
4       1    13.24        2.59  2.87  21.0  118     2.80        2.69
..    ...      ...         ...   ...   ...  ...      ...         ...
173     3    13.71        5.65  2.45  20.5   95     1.68        0.61
174     3    13.40        3.91  2.48  23.0  102     1.80        0.75
175     3    13.27        4.28  2.26  20.0  120     1.59        0.69
176     3    13.17        2.59  2.37  20.0  120     1.65        0.68
177     3    14.13        4.10  2.74  24.5   96     2.05        0.76

     Nonflavanoid.phenols  Proanth  Color.int   Hue    OD  Proline
0                    0.28     2.29       5.64  1.04  3.92     1065
1                    0.26     1.28       4.38  1.05  3.40     1050
2                    0.30     2.81       5.68  1.03  3.17     1185
3                    0.24     2.18       7.80  0.86  3.45     1480
4                    0.39     1.82       4.32  1.04  2.93      735
..                    ...      ...        ...   ...   ...      ...
173                  0.52     1.06       7.70  0.64  1.74      740
174                  0.43     1.41       7.30  0.70  1.56      750
175                  0.43     1.35      10.20  0.59  1.56      835
176                  0.53     1.46       9.30  0.60  1.62      840
177                  0.56     1.35       9.20  0.61  1.60      560

[178 rows x 14 columns]
```

Input:

```
df1 = pd.read_csv(r"wine.csv", usecols=[0, 1, 2], skiprows=1)
df1.columns = ['classlabel', 'Alcohol', 'Malic Acid']
print("Original DataFrame:")
df1
```

Output:

```
Original DataFrame:

     classlabel  Alcohol  Malic Acid
0             1    13.20        1.78
1             1    13.16        2.36
2             1    14.37        1.95
3             1    13.24        2.59
4             1    14.20        1.76
..          ...      ...         ...
172           3    13.71        5.65
173           3    13.40        3.91
174           3    13.27        4.28
175           3    13.17        2.59
176           3    14.13        4.10

[177 rows x 3 columns]
```

Input:

```
scaling=MinMaxScaler()
scaled_value=scaling.fit_transform(df1[['Alcohol','Malic Acid']])
df1[['Alcohol','Malic Acid']]=scaled_value
print("\n Dataframe after MinMax Scaling")
df1
```

Output:

```
Dataframe after MinMax Scaling

     classlabel   Alcohol  Malic Acid
0             1  0.571053    0.205534
1             1  0.560526    0.320158
2             1  0.878947    0.239130
3             1  0.581579    0.365613
4             1  0.834211    0.201581
..          ...       ...         ...
172           3  0.705263    0.970356
173           3  0.623684    0.626482
174           3  0.589474    0.699605
175           3  0.563158    0.365613
176           3  0.815789    0.664032

[177 rows x 3 columns]
```

Input:

```
scaling=StandardScaler()
scaled_standardvalue=scaling.fit_transform(df1[['Alcohol','Malic Acid']])
df1[['Alcohol','Malic Acid']]=scaled_standardvalue
print("\n Dataframe after Standard Scaling")
df1
```

Output:

```
Dataframe after Standard Scaling


     classlabel   Alcohol  Malic Acid
0             1  0.255824   -0.501624
1             1  0.206229    0.018020
2             1  1.706501   -0.349315
3             1  0.305420    0.224086
4             1  1.495719   -0.519543
..          …        …          …
172           3  0.888171    2.965658
173           3  0.503803    1.406725
174           3  0.342617    1.738222
175           3  0.218628    0.224086
176           3  1.408926    1.576953

[177 rows x 3 columns]
```

Input:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
iris=pd.read_csv(r"Iris.csv")
iris
```

Output:

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   …            …           …            …           …
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            Species
0       Iris-setosa
1       Iris-setosa
2       Iris-setosa
3       Iris-setosa
4       Iris-setosa
..              …
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

[150 rows x 6 columns]
```

Input:

```
le=LabelEncoder()
iris['code']=le.fit_transform(iris.Species)
iris
```

Output:

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   …            …             …              …             …
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            Species  code
0       Iris-setosa     0
1       Iris-setosa     0
2       Iris-setosa     0
3       Iris-setosa     0
4       Iris-setosa     0
..              …   …
145  Iris-virginica     2
146  Iris-virginica     2
147  Iris-virginica     2
148  Iris-virginica     2
149  Iris-virginica     2

[150 rows x 7 columns]
```

Practical 4

Aim: Hypothesis Testing

- Formulate null and alternative hypotheses for a given problem.
- Conduct a hypothesis test using appropriate statistical tests (e.g., t-test, chi- square test).
- Interpret the results and draw conclusions based on the test outcomes.

Hypothesis testing is a statistical method used to make decisions about a population based on sample data. The **Null Hypothesis (H_0)** assumes no effect or no difference, while the **Alternative Hypothesis (H_1)** represents the presence of an effect or difference. Data is analyzed to decide whether to reject or fail to reject H_0. This approach provides an objective framework for decision-making.

A **hypothesis test** is a step-by-step statistical procedure used to evaluate assumptions about a population parameter. It involves defining hypotheses, selecting a significance level, calculating a test statistic, and making a decision based on the p-value or critical value. Hypothesis tests help determine whether observed results are statistically significant. They reduce uncertainty in data-driven conclusions.

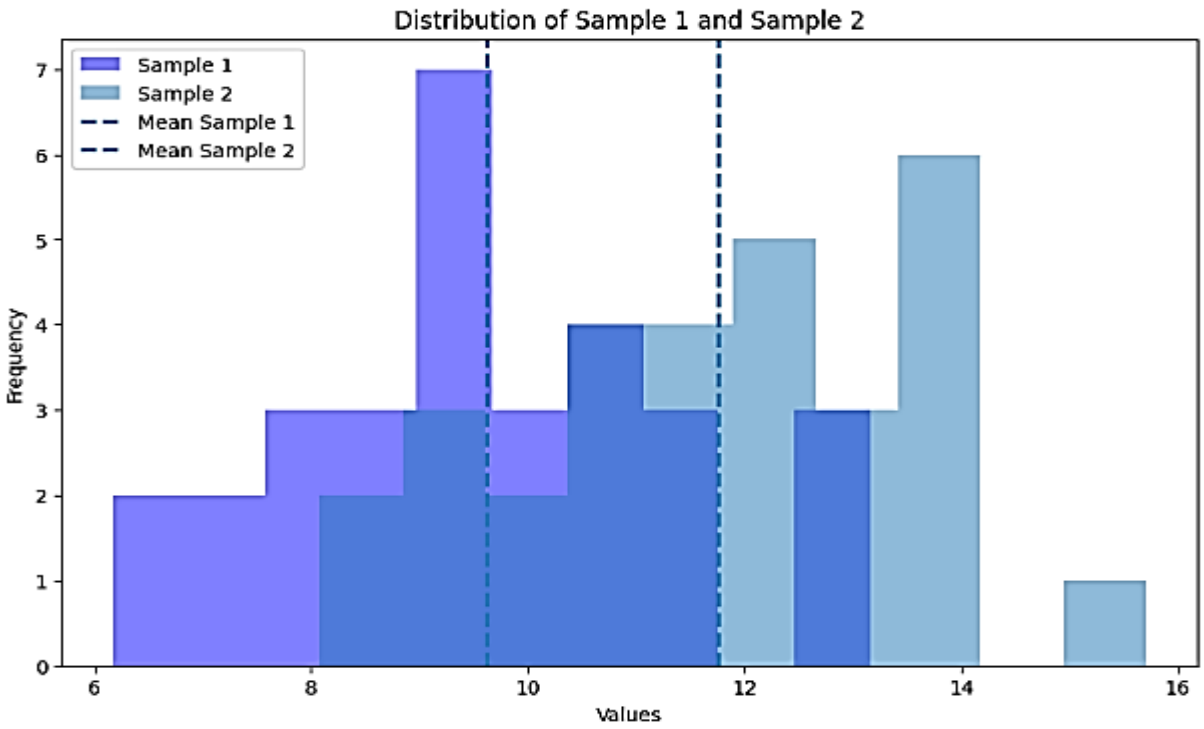
A **statistical test** is a mathematical method used to compare sample data against a hypothesis or between groups. Common statistical tests include **t-test, z-test, chi-square test, and ANOVA**. Each test is chosen based on data type, sample size, and study objective. Statistical tests quantify evidence against the null hypothesis.

Input:

```
# t-test
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

# Generate two samples for demonstrate purposes
np.random.seed(42)
sample1 = np.random.normal(loc=10, scale=2, size=30)
sample2 = np.random.normal(loc=12, scale=2, size=30)
# Perform a two-sample t-test
t_statistic, p_value = stats.ttest_ind(sample1, sample2)

# Set the signifiacnt level
alpha = 0.05
print("Results of Two-sample t-test:")
print(f'T-statistic:{t_statistic}')
print(f'P-value:{p_value}')
print(f"Degree of Freedom: {len(sample1) + len(sample2)-2}")

# Plot the distributions
plt.figure(figsize = (10, 6))
plt.hist(sample1, alpha = 0.5, label = 'Sample 1', color = 'blue')
plt.hist(sample2, alpha=0.5, label='Sample 2')

plt.axvline(np.mean(sample1), linestyle='--', linewidth=2, label='Mean Sample 1')
plt.axvline(np.mean(sample2), linestyle='--', linewidth=2, label='Mean Sample 2')

plt.title('Distribution of Sample 1 and Sample 2')
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

Output:

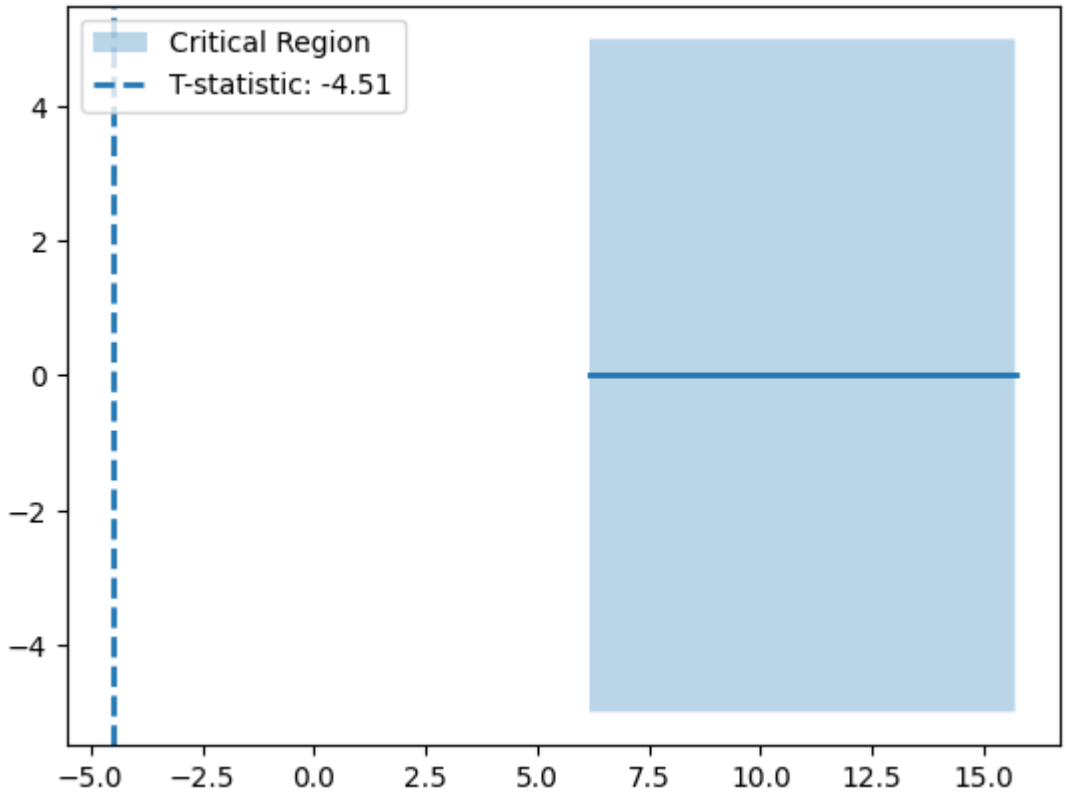
```
Results of Two-sample t-test:
T-statistic:-4.512913234547555
P-value:3.176506547470154e-05
Degree of Freedom: 58
```



Input:

```
# Highlight the critical region if null hypothesis is rejected
if p_value < alpha:
    critical_region = np.linspace(
        min(sample1.min(), sample2.min()),
        max(sample1.max(), sample2.max()),
        1000
    )
    # Draw a baseline instead of an empty canvas
    plt.plot(critical_region, np.zeros_like(critical_region), linewidth=2)

    # Highlight critical region as a narrow band
    plt.fill_between(
        critical_region,
        -5,
        5,
        alpha=0.3,
        label='Critical Region'
    )

    # Mark the t-statistic clearly
    plt.axvline(
        t_statistic,
        linestyle='--',
        linewidth=2,
        label=f"T-statistic: {t_statistic:.2f}"
    )
    plt.legend()
plt.show()
```

Output:



Input:

```
# Draw Conclusions
if p_value < alpha:
    if np.mean(sample1) > np.mean(sample2):
        print("Conclusion: There is significant evidence to reject the null hypothesis.")
        print("Interpretation: The mean of Sample 1 is significantly higher than that of Sample 2.")
    else:
        print("Conclusion: There is significant evidence to reject the null hypothesis.")
        print("Interpretation: The mean of Sample 2 is significantly higher than that of Sample 1.")
else:
    print("Conclusion: Fail to reject the null hypothesis.")
    print("Interpretation: There is not enough evidence to claim a significant difference between the means.")
```

Output:

```
Conclusion: There is significant evidence to reject the null hypothesis.
Interpretation: The mean of Sample 2 is significantly higher than that of Sample
1.
```

Input:

```
#chi-test

import pandas as pd
import numpy as np
import matplotlib as plt
import seaborn as sb
import warnings

from scipy import stats

warnings.filterwarnings('ignore')

df=sb.load_dataset('mpg')
print(df)

print(df['horsepower'].describe())
```

```
print(df['model_year'].describe())

bins=[0,75,150,240]
df['horsepower_new']=pd.cut(df['horsepower'],bins=bins,labels=['l','m','h'])
c=df['horsepower_new']
print(c)

ybins=[69,72,74,84]
label=['t1','t2','t3']
df['modelyear_new']=pd.cut(df['model_year'],bins=ybins,labels=label)
newyear=df['modelyear_new']
print(newyear)

df_chi=pd.crosstab(df['horsepower_new'],df['modelyear_new'])
print(df_chi)

print(stats.chi2_contingency(df_chi))
```

Output:

```
      mpg  cylinders  displacement  horsepower  weight  acceleration  \
0    18.0          8         307.0       130.0    3504          12.0
1    15.0          8         350.0       165.0    3693          11.5
2    18.0          8         318.0       150.0    3436          11.0
3    16.0          8         304.0       150.0    3433          12.0
4    17.0          8         302.0       140.0    3449          10.5
..    ...        ...           ...         ...     ...           ...
393  27.0          4         140.0        86.0    2790          15.6
394  44.0          4          97.0        52.0    2130          24.6
395  32.0          4         135.0        84.0    2295          11.6
396  28.0          4         120.0        79.0    2625          18.6
397  31.0          4         119.0        82.0    2720          19.4

     model_year  origin                       name
```

```
0            70     usa  chevrolet chevelle malibu
1            70     usa          buick skylark 320
2            70     usa         plymouth satellite
3            70     usa              amc rebel sst
4            70     usa                ford torino
..          ...     ...                        ...
393          82     usa            ford mustang gl
394          82  europe                  vw pickup
395          82     usa              dodge rampage
396          82     usa                ford ranger
397          82     usa                 chevy s-10

[398 rows x 9 columns]
count    392.000000
mean     104.469388
std       38.491160
min       46.000000
25%       75.000000
50%       93.500000
75%      126.000000
max      230.000000
Name: horsepower, dtype: float64
count    398.000000
mean      76.010050
std        3.697627
min       70.000000
25%       73.000000
50%       76.000000
75%       79.000000
max       82.000000
Name: model_year, dtype: float64
0      m
1      h
2      m
3      m
4      m
      ..
393    m
394    l
395    m
396    m
397    m
Name: horsepower_new, Length: 398, dtype: category
Categories (3, object): ['l' < 'm' < 'h']
0      t1
1      t1
2      t1
3      t1
```

```
4      t1
       ..
393    t3
394    t3
395    t3
396    t3
397    t3
Name: modelyear_new, Length: 398, dtype: category
Categories (3, object): ['t1' < 't2' < 't3']
modelyear_new   t1  t2   t3
horsepower_new
l                9  14   76
m               49  41  158
h               26  11    8
Chi2ContingencyResult(statistic=np.float64(54.95485392447537),
pvalue=np.float64(3.32051800955598 4e-11), dof=4, expected_freq=array([[
21.21428571,  16.66836735,  61.11734694],
       [ 53.14285714,  41.75510204, 153.10204082],
       [  9.64285714,   7.57653061,  27.78061224]]))
```

Practical 5

Aim: ANOVA (Analysis of Variance)

- Perform one-way ANOVA to compare means across multiple groups.
- Conduct post-hoc tests to identify significant differences between group means.

One-way ANOVA (Analysis of Variance) is a statistical technique used to compare the means of **three or more independent groups** based on one factor. It tests whether at least one group mean is significantly different from the others. ANOVA works by analyzing variance within groups and between groups. It helps avoid multiple t-tests and reduces error.
A **post-hoc test** is performed **after ANOVA** when the null hypothesis is rejected. It identifies **which specific group pairs differ significantly**. Common post-hoc tests include **Tukey's HSD, Bonferroni, and Scheffé tests**. Post-hoc analysis controls the risk of false positives caused by multiple comparisons.

Input:

```
import pandas as pd
from scipy.stats import f_oneway
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# Step 1: Load Dataset
df = pd.read_csv("Iris.csv")
print(df.head())
```

Output:

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

Input:

```
# Step 2: Separate Groups
# Using SepalLengthCm across Species
setosa = df[df['Species'] == 'Iris-setosa']['SepalLengthCm']
versicolor = df[df['Species'] == 'Iris-versicolor']['SepalLengthCm']
virginica = df[df['Species'] == 'Iris-virginica']['SepalLengthCm']

# Step 3: One-Way ANOVA
f_statistic, p_value = f_oneway(setosa, versicolor, virginica)

print("\nOne-Way ANOVA Result")
print("F-statistic:", f_statistic)
print("P-value:", p_value)

# Step 4: Decision
if p_value < 0.05:
    print("Result: Reject Null Hypothesis (Significant Difference Exists)")
else:
    print("Result: Fail to Reject Null Hypothesis (No Significant Difference)")

# Step 5: Post-Hoc Test (Tukey HSD)
tukey = pairwise_tukeyhsd(
    endog=df['SepalLengthCm'],
    groups=df['Species'],
    alpha=0.05
)

print("\nPost-Hoc Test (Tukey HSD) Results:")
print(tukey)
```

Output:

```
One-Way ANOVA Result
F-statistic: 119.26450218450468
P-value: 1.669669190769383e-31
Result: Reject Null Hypothesis (Significant Difference Exists)

Post-Hoc Test (Tukey HSD) Results:
        Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====================================================================
     group1          group2     meandiff p-adj lower  upper  reject
---------------------------------------------------------------------
    Iris-setosa Iris-versicolor     0.93   0.0 0.6862 1.1738   True
    Iris-setosa  Iris-virginica    1.582   0.0 1.3382 1.8258   True
Iris-versicolor  Iris-virginica    0.652   0.0 0.4082 0.8958   True
---------------------------------------------------------------------
```

Practical 6

Aim: Regression and Its Types

- Implement simple linear regression using a dataset.
- Explore and interpret the regression model coefficients and goodness-of-fit measures.
- Extend the analysis to multiple linear regression and assess the impact of additional predictors.

Regression is a technique used to find the relationship between a dependent variable and one or more independent variables. It is mainly used for prediction and trend analysis.
Simple linear regression models the relationship between one independent variable and one dependent variable using a straight line.
Multiple linear regression predicts a dependent variable using two or more independent variables. It analyzes the combined effect of all predictors on the output.
Polynomial regression models non-linear relationships by using polynomial terms of the independent variable. It is used when data does not fit a straight line.

Input:

```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
df = pd.read_csv(r"housing.csv")
df.head()
```

Output:

```
   Avg. Area Income  Avg. Area House Age  Avg. Area Number of Rooms  \
0       79545.45857             5.682861                   7.009188
1       79248.64245             6.002900                   6.730821
2       61287.06718             5.865890                   8.512727
3       63345.24005             7.188236                   5.586729
4       59982.19723             5.040555                   7.839388

   Avg. Area Number of Bedrooms  Area Population         Price  \
0                          4.09      23086.80050  1.059034e+06
1                          3.09      40173.07217  1.505891e+06
2                          5.13      36882.15940  1.058988e+06
3                          3.26      34310.24283  1.260617e+06
4                          4.23      26354.10947  6.309435e+05

                                            Address
0  208 Michael Ferry Apt. 674\nLaurabury, NE 3701…
1  188 Johnson Views Suite 079\nLake Kathleen, CA…
2  9127 Elizabeth Stravenue\nDanieltown, WI 06482…
3                         USS Barnett\nFPO AP 44820
4                        USNS Raymond\nFPO AE 09386
```

Input:

```
df.tail()
```

Output:

```
      Avg. Area Income  Avg. Area House Age  Avg. Area Number of Rooms  \
4995       60567.94414             7.830362                   6.137356
4996       78491.27543             6.999135                   6.576763
4997       63390.68689             7.250591                   4.805081
4998       68001.33124             5.534388                   7.130144
4999       65510.58180             5.992305                   6.792336
```

Input:

df.shape

Output:

```
(5000, 7)
```

Input:

df.size

Output:

```
35000
```

Input:

df.describe()

Output:

```
       Avg. Area Income  Avg. Area House Age  Avg. Area Number of Rooms  \
count       5000.000000          5000.000000                5000.000000
mean       68583.108984             5.977222                   6.987792
std        10657.991214             0.991456                   1.005833
min        17796.631190             2.644304                   3.236194
25%        61480.562390             5.322283                   6.299250
50%        68804.286405             5.970429                   7.002902
75%        75783.338665             6.650808                   7.665871
max       107701.748400             9.519088                  10.759588

       Avg. Area Number of Bedrooms  Area Population         Price
count                   5000.000000      5000.000000  5.000000e+03
mean                       3.981330     36163.516039  1.232073e+06
std                        1.234137      9925.650114  3.531176e+05
min                        2.000000       172.610686  1.593866e+04
25%                        3.140000     29403.928700  9.975771e+05
50%                        4.050000     36199.406690  1.232669e+06
75%                        4.490000     42861.290770  1.471210e+06
max                        6.500000     69621.713380  2.469066e+06
```

Input:

df.info()

Output:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Avg. Area Income              5000 non-null   float64
 1   Avg. Area House Age           5000 non-null   float64
 2   Avg. Area Number of Rooms     5000 non-null   float64
 3   Avg. Area Number of Bedrooms  5000 non-null   float64
 4   Area Population               5000 non-null   float64
 5   Price                         5000 non-null   float64
 6   Address                       5000 non-null   object
dtypes: float64(6), object(1)
memory usage: 273.6+ KB
```

Input:

```
df.dtypes
```

Output:

```
Avg. Area Income                float64
Avg. Area House Age             float64
Avg. Area Number of Rooms       float64
Avg. Area Number of Bedrooms    float64
Area Population                 float64
Price                           float64
Address                          object
dtype: object
```

Input:

```
#import ssl
#ssl._create_default_https_context = ssl._create_unverified_context
housing = fetch_california_housing()
# Convert to DataFrame
housing_df = pd.DataFrame(housing.data, columns=housing.feature_names)
housing_df.head() # Print first few rows
```

Output:

```
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude
0    -122.23
1    -122.22
2    -122.24
3    -122.25
4    -122.25
```

Input:

```
housing_df['PRICE']=housing.target
X=housing_df[['AveRooms']]
y=housing_df[['PRICE']]
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
model=LinearRegression()
model.fit(X_train,y_train)
```

Output:

```
LinearRegression()
```

Input:

```
mse=mean_squared_error(y_test,model.predict(X_test))
r2=r2_score(y_test,model.predict(X_test))
print("Mean Squared Error: ", mse)
print("R-squared: ",r2)
print("Intercept: ",model.intercept_)
print("Co-efficient: ",model.coef_)
```

Output:

```
Mean Squared Error:  1.2923314440807299
R-squared:  0.013795337532284901
Intercept:  [1.65476227]
Co-efficient:  [[0.07675559]]
```

Input:

```
X = housing_df.drop('PRICE',axis=1)
y = housing_df['PRICE']
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
model = LinearRegression()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test,y_pred)
r2 = r2_score(y_test,y_pred)
print("Mean Squared Error:",mse)
print("R-squared:",r2)
print("Intercept:",model.intercept_)
print("Coefficient:",model.coef_)
```

Output:

```
Mean Squared Error: 0.5558915986952442
R-squared: 0.575787706032451
Intercept: -37.02327770606416
Coefficient: [ 4.48674910e-01  9.72425752e-03 -1.23323343e-01  7.83144907e-01
 -2.02962058e-06 -3.52631849e-03 -4.19792487e-01 -4.33708065e-01]
```

Practical 7

Aim: Logistic Regression and Decision Tree

- Build a logistic regression model to predict a binary outcome.
- Evaluate the model's performance using classification metrics (e.g., accuracy, precision, recall).
- Construct a decision tree model and interpret the decision rules for classification.

Logistic Regression is a classification algorithm used to predict **binary outcomes** such as yes/no or 0/1. It uses a **sigmoid function** to convert linear output into probabilities. It is widely used in classification problems.

Input:

```
import numpy as numpy
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

#Load dataset
df = pd.read_csv("Iris-1.csv")
df = df[df['Species'] != 'Iris-virginica']

X = df.drop('Species', axis =1)
y = df['Species']

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size = 0.2, random_state = 42
)

logistic_model = LogisticRegression()
logistic_model.fit(X_train, y_train)

# Predictions
y_pred_logistic = logistic_model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred_logistic))
print("\nClassification Report")
print(classification_report(y_test, y_pred_logistic))
```

Output:

```
Accuracy: 1.0

Classification Report
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        12
Iris-versicolor       1.00      1.00      1.00         8

       accuracy                           1.00        20
      macro avg       1.00      1.00      1.00        20
   weighted avg       1.00      1.00      1.00        20
```

A **Decision Tree** is a supervised learning algorithm used for **classification and regression**. It works by splitting data into branches based on feature conditions. The final decision is made at leaf nodes, making the model easy to interpret.

Input:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create Model
model = DecisionTreeClassifier()

# Train Model
model.fit(X_train, y_train)

# Predictions
y_pred_tree = model.predict(X_test)

# Evaluation
print("\nDecision Tree Metrics")
print("Accuracy: ", accuracy_score(y_test, y_pred_tree))
print("\nClassification Report")
print(classification_report(y_test, y_pred_tree))
```

Output:

```
Decision Tree Metrics
Accuracy:  1.0

Classification Report
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        12
Iris-versicolor       1.00      1.00      1.00         8

       accuracy                           1.00        20
      macro avg       1.00      1.00      1.00        20
   weighted avg       1.00      1.00      1.00        20
```

Practical 8

Aim: K-Means Clustering

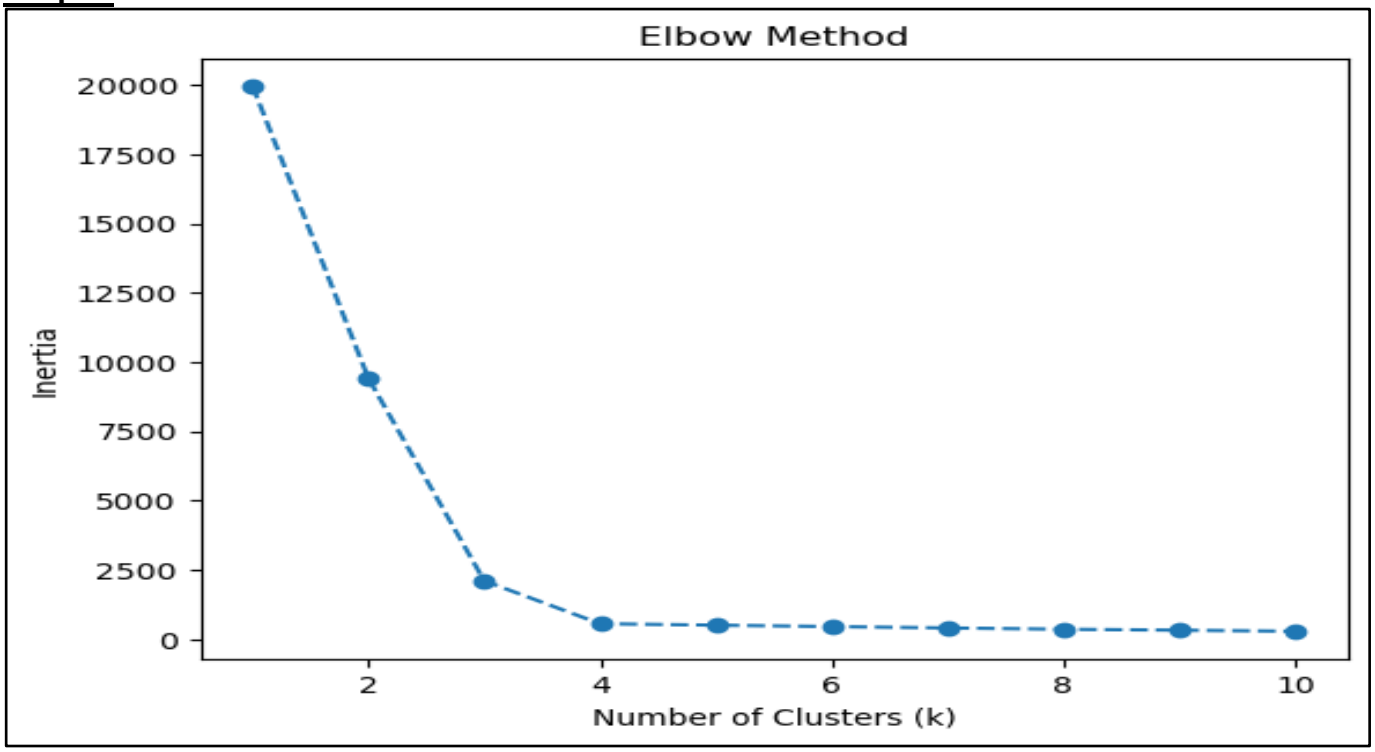
- Apply the K-Means algorithm to group similar data points into clusters.
- Determine the optimal number of clusters using elbow method or silhouette analysis.
- Visualize the clustering results and analyze the cluster characteristics.

K-Means is an **unsupervised learning algorithm** used to group data into **K clusters** based on similarity. It works by assigning data points to the nearest cluster centroid and updating centroids iteratively. The algorithm aims to minimize within-cluster variance.

Input:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
# Generate synthetic data
X, _ = make_blobs(n_samples=300, centers=4, random_state=42)
# Step 1: Elbow Method to find the optimal number of clusters
inertia = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)
K_range = range(1, 11)
inertia = []
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)
plt.plot(K_range, inertia, marker='o', linestyle='--')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

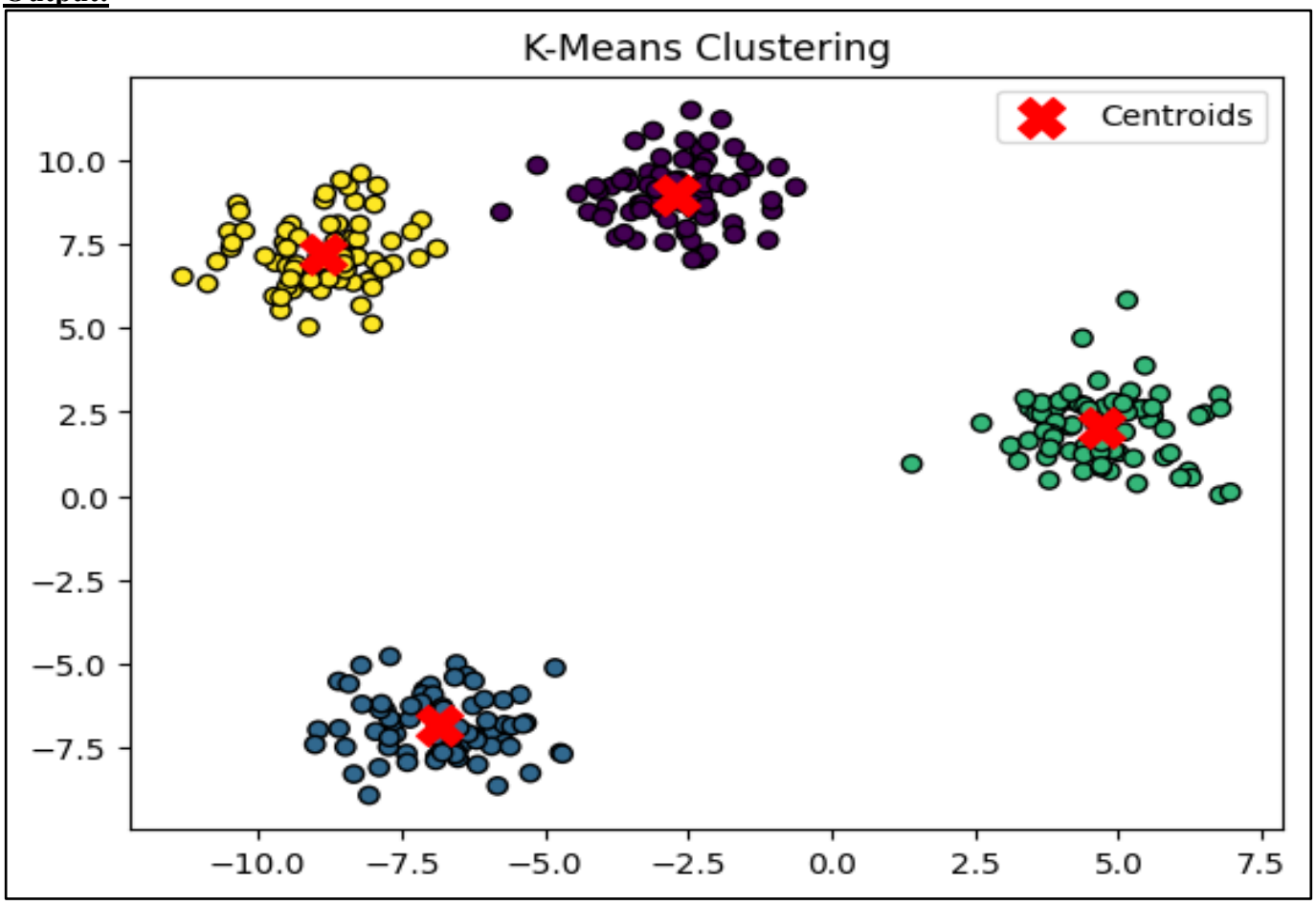
Output:



Input:

```
# Step 2: Apply K-Means with the chosen k (let's pick k=4)
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X)
# Step 3: Visualize Clustering Results
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='viridis', edgecolors='k')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
 s=200, c='red', marker='X', label='Centroids')
plt.title('K-Means Clustering')
plt.legend()
plt.show()
```

Output:



Practical 9

<u>Aim:</u> Principal Component Analysis (PCA)

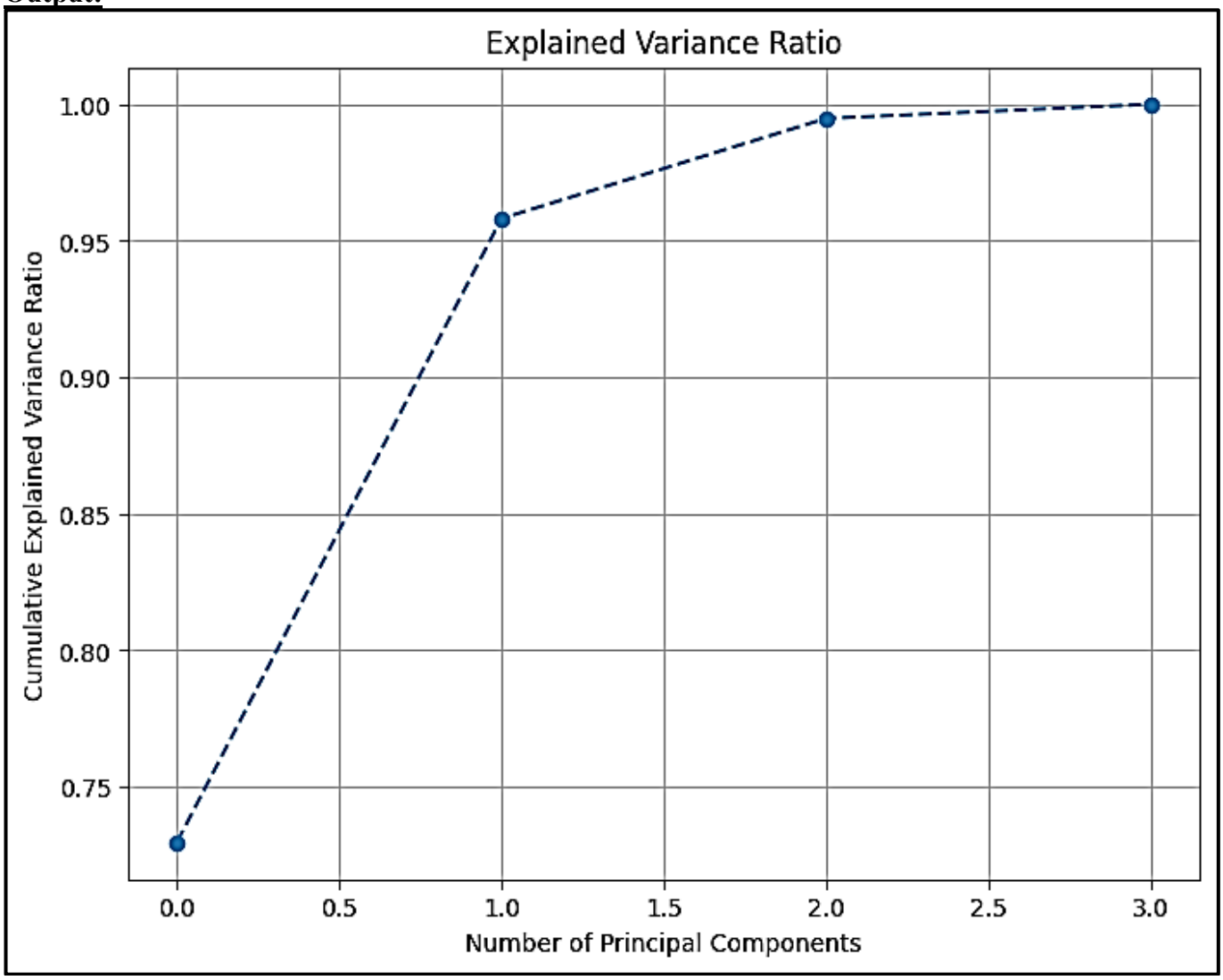
- Perform PCA on a dataset to reduce dimensionality.
- Evaluate the explained variance and select the appropriate number of principal components.
- Visualize the data in the reduced-dimensional space.

Principal Component Analysis (PCA) is a **dimensionality reduction technique** used to transform high-dimensional data into a smaller set of uncorrelated variables called **principal components**. It preserves maximum variance while reducing features. PCA improves performance and visualization of data.

<u>Input:</u>

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# Load the Iris dataset
iris = load_iris()

iris_df = pd.DataFrame(
    data=np.c_[iris['data'], iris['target']],
    columns=iris['feature_names'] + ['target']
)
# Features and target
X = iris_df.drop('target', axis=1)
y = iris_df['target']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA()
X_pca = pca.fit_transform(X_scaled)
explained_variance_ratio = pca.explained_variance_ratio_
plt.figure(figsize=(8, 6))
plt.plot(
    np.cumsum(explained_variance_ratio),
    marker='o',
    linestyle='--'
)
plt.title('Explained Variance Ratio')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance Ratio')
plt.grid(True)
plt.show()
```

Output:



Input:

```
cumulative_variance_ratio = np.cumsum(explained_variance_ratio)
n_components = np.argmax(cumulative_variance_ratio >= 0.95) + 1
print(f'Number of principal components to explain 95% variance: {n_components}')
```

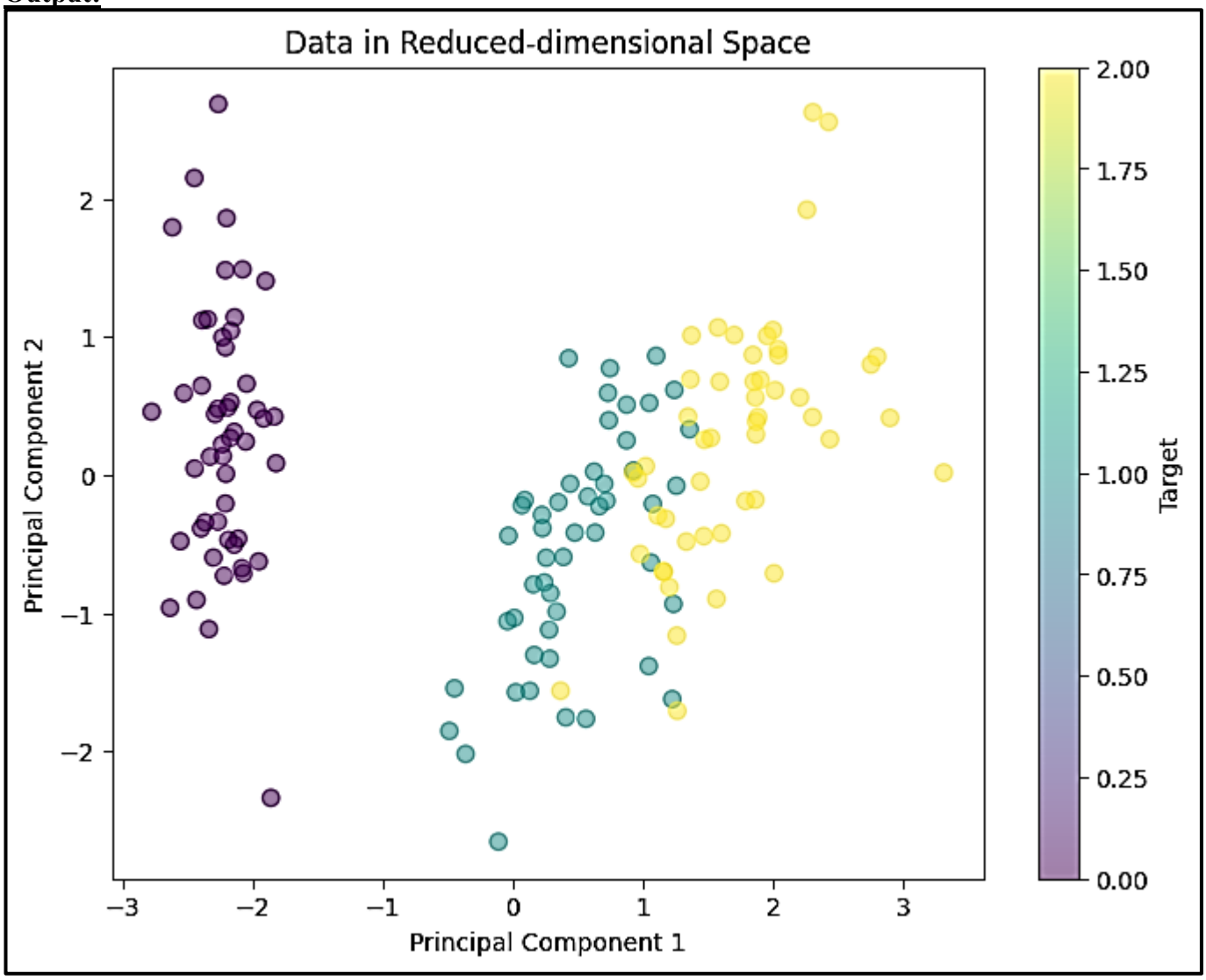
Output:

```
Number of principal components to explain 95% variance: 2
```

Input:

```
pca = PCA(n_components=n_components)
X_reduced = pca.fit_transform(X_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(
    X_reduced[:, 0],
    X_reduced[:, 1],
    c=y,
    cmap='viridis',
    s=50,
    alpha=0.5
)
plt.title('Data in Reduced-dimensional Space')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(label='Target')
plt.show()
```

Output:


Data in Reduced-dimensional Space
Principal Component 2
Principal Component 1
Target
2
1
0
−1
−2
−3
−2
−1
0
1
2
3
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00

Practical 10

Aim: Data Visualization and Storytelling

- Create meaningful visualizations using data visualization tools.
- Combine multiple visualizations to tell a compelling data story.
- Present the findings and insights in a clear and concise manner.

Data visualization is the process of representing data visually using charts, graphs, and dashboards to simplify complex information.

Data storytelling combines visualizations with context, narrative, and insights to explain **what happened, why it happened, and what to do next**. It helps decision-makers understand trends and patterns quickly. Effective storytelling improves communication and impact of data analysis.

Input:

```
import pandas as pd
import matplotlib.pyplot as plt
# Step 1: Load Dataset
df = pd.read_csv("Iris.csv")
print(df.head())
```

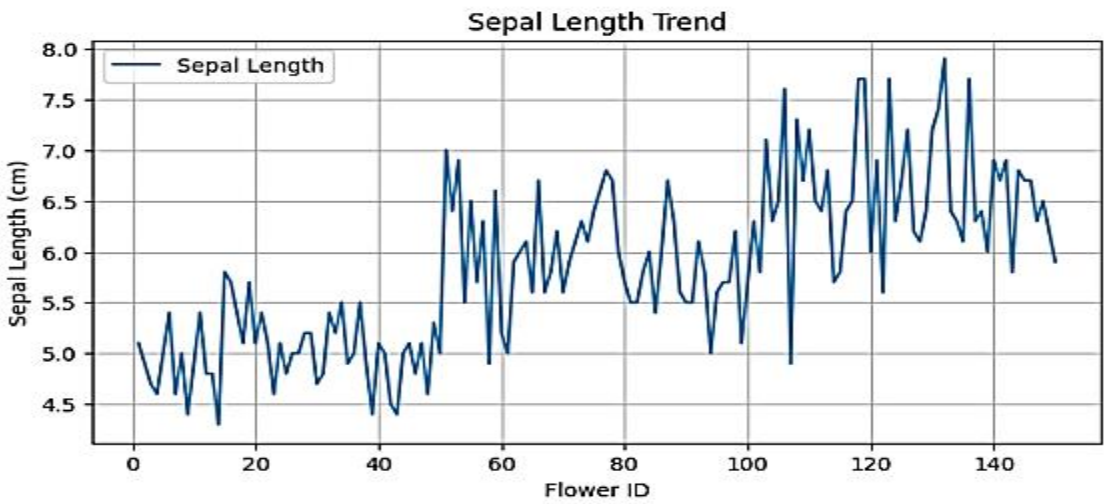
Output:

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

Input:

```
# Step 2: Visualization 1
# Line Plot – Sepal Length Trend
plt.figure(figsize=(8, 4))
plt.plot(df['Id'], df['SepalLengthCm'], label='Sepal Length')
plt.title("Sepal Length Trend")
plt.xlabel("Flower ID")
plt.ylabel("Sepal Length (cm)")
plt.grid(True)
plt.legend()
plt.show()
```

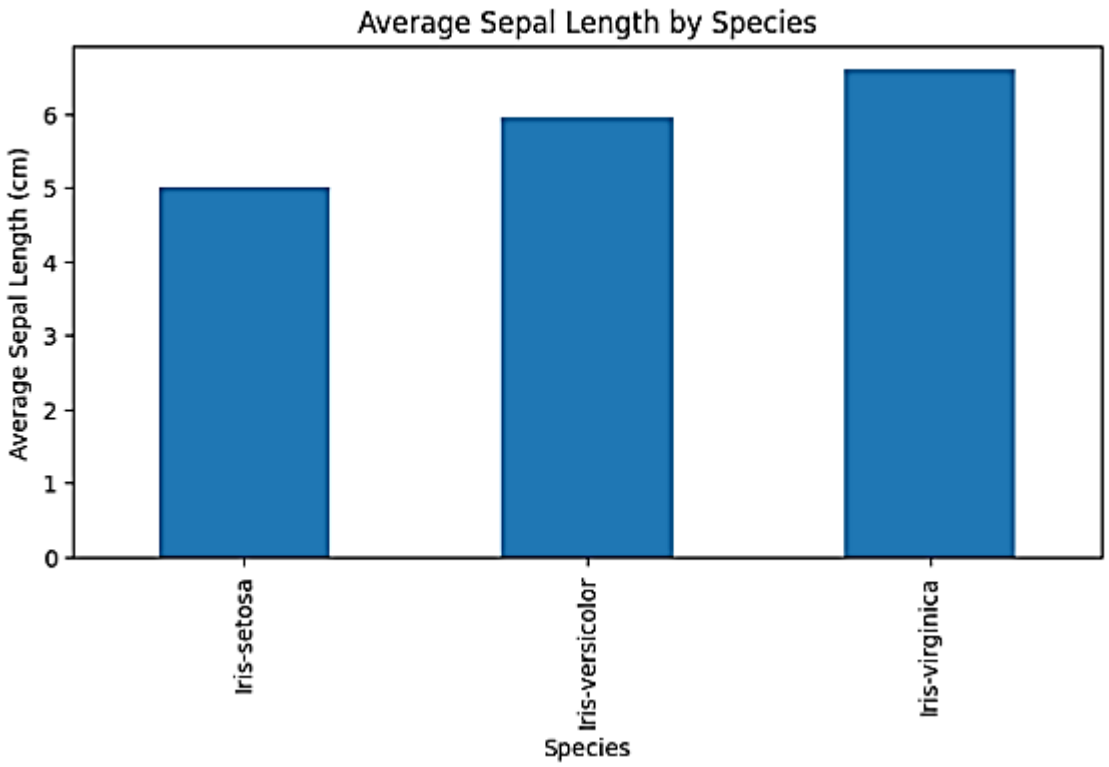
Output:



Input:

```
# Step 3: Visualization 2
# Bar Chart – Average Sepal Length by Species
avg_sepal = df.groupby('Species')['SepalLengthCm'].mean()
plt.figure(figsize=(8, 4))
avg_sepal.plot(kind='bar')
plt.title("Average Sepal Length by Species")
plt.xlabel("Species")
plt.ylabel("Average Sepal Length (cm)")
plt.show()
```

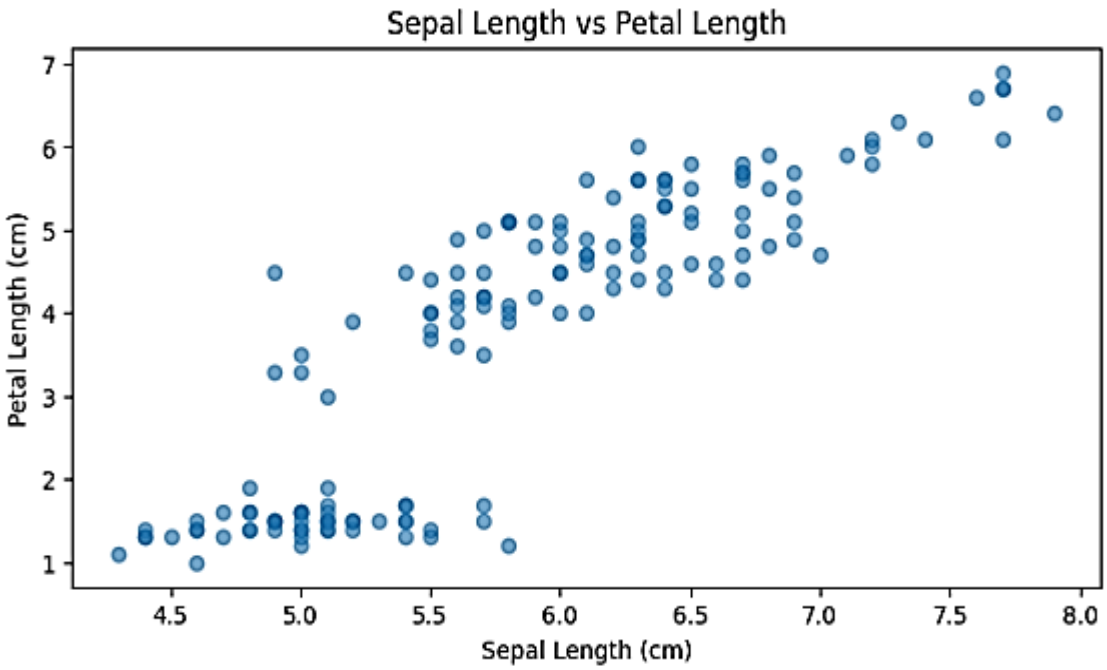
Output:



Input:

```
# Step 4: Visualization 3
# Scatter Plot – Sepal Length vs Petal Length
plt.figure(figsize=(8, 4))
plt.scatter(df['SepalLengthCm'], df['PetalLengthCm'], alpha=0.6)
plt.title("Sepal Length vs Petal Length")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.show()
```

Output:



Input:

```
# Step 5: Storytelling – Insights
print("\nKey Insights from Data Visualization:")
print("1. Sepal length shows variation across different flowers.")
print("2. Iris-virginica has the highest average sepal length.")
print("3. Strong positive relationship exists between sepal and petal length.")
print("4. Species-wise comparison helps in understanding flower characteristics.")
```

Output:

```
Key Insights from Data Visualization:
1. Sepal length shows variation across different flowers.
2. Iris-virginica has the highest average sepal length.
3. Strong positive relationship exists between sepal and petal length.
4. Species-wise comparison helps in understanding flower characteristics.
```